

プロジェクト演習 14 Linux プログラミング

引田・山崎

2025 年度 第 1 版

この授業について

これまで授業では、ほとんど Windows を使ってきた。Windows 以外にもさまざまな OS があるが、中でも特に多く利用されているものが UNIX である。UNIX とは（現在においては）ある特定の一つの OS のことではない。さまざまな種類があり、この授業で取り上げる Linux も UNIX の一つである。例えば、Apple 社の Macintosh 用の OS (macOS) も UNIX の一つである。ちなみに PlayStation や Switch は FreeBSD (UNIX の一つ) と言われている。

Linux は、組み込みシステムから大規模なサーバーまで、さまざまに利用されている。この授業では、Linux を組み込んだシステムの開発方法を理解し、自分でも簡単な開発ができるようになることを目標とする。

より具体的な到達目標は、以下の通りである。

- Linux がそれなりに使える (特に CUI に慣れる)
- shell スクリプトが作れる
- 並行プログラミングの基本を理解しプログラムを作成できる
- Linux 組み込みシステムのクロス開発環境を理解しプログラムを作成できる
- 通信プログラミング (ソケット) の基本を理解しプログラムを作成できる

「プログラミング」などの授業との違いは何か? これまでは、プログラミング言語を習得するための授業だった。言語を習得するだけでプログラミングできるような問題を解いてきた。例えば、文字の表示は、`printf` だけを知っていれば十分だった。

しかし、実際のプログラミングはまったく違う。多くの場合、一つのシステムの中のある一部分を作っていくことになる。OS を呼び出してグラフィックスを描いたり、他のアプリケーションとの間で文字をコピー・ペースト可能にしたり、他のアプリケーションと同時並行に動作したりといったことが求められる。このためには、システムの全体の基本的な動作を習得し、必要な API (Application Program Interface) を見つけ、その利用法を理解することに非常に多くの時間を費やす。

また、他人の書いたプログラムを読むということもよくある。現在においては、何かやりたいと思ったときに、助けになる元のプログラムが存在しないなどということはほとんどない。その分自分は楽になるのだが、その代わりそのプログラムを読まなければならない。今回は、山崎の書いたプログラムを理解し、それをアレンジしていくことで、プログラムを完成させる。

更に、現在ではさまざまなツールがあり、個々の作業を簡単に行うことができる。しかし、何のために何を使うのが効果的かをよく考えないと、作業効率がかえって悪化する。目的意識をもって、自分で作業環境を工夫していくことも、とても重要である。

今回は、組み込み開発環境の中でソフトウェアを作ることで、上記のすべてを演習していく。

授業の受け方: この授業は、会社で先輩や上司から話を聞いたり教わったりしながら、仕事を進めていくことをイメージしている。すべてを口頭で説明できないので文書になっているが、故意に情報を散在させたり、情報を不足させたり、一回説明したことを大幅に省略したりしている。(■で書いた部分はこの資料とは別に教員が講義をする部分。)

ノートをつける：先輩 (TA) や上司 (教員) に同じことを 2 回聞かないようにする (これも訓練)。そこで自分でノートを作る。社会に出たら、きちんとノートを作ることは必須である。作業の意味を考えながら、どのようにまとめたら良いかも考える。

グループについて：この授業は、前述のように新たに習得すべき技術や調べることがとても多いため、グループ内で相談したり手分けしたりしても構わない。しかし、プログラムやレポートは一人で作成する。プロジェクト演習は 3 年間の中での最重要科目であり、全力投入して欲しい。(当然、レポートの丸写し等があった場合は、単位取り消し。)

前提となる知識：

C プログラミングについては、少なくとも「情報処理演習」を理解していないと無理である。「プログラミング 1」の構造体、「プログラミング 2」のポイントまで理解していた方が望ましいが、一部にしか出てこない。忘れてしまった者や履修していない者はしっかり自習すること。この演習が完了すれば、社会に出てようやく C のプログラミングができると言っても恥ずかしくないレベルになるので頑張ってもらいたい。

また、Linux の最低限の知識は説明するが、上にも書いたように自分が実行したコマンドの意味を一つ一つ確認してノートを作っていくこと。

備え付けの書籍：この演習では自分で調べることが重要である。Web も便利だが、そもそも何を調べたら良いかも分からない状態のときは、本を読んで理解するしかない。この演習用に、以下の書籍を本棚に用意している。

- 武藤: Debian GNU/Linux 徹底入門第 3 版, 翔泳社 [4 冊]
Debian という Linux の基本的な使い方と管理方法を紹介した本。古い本だが基本的なところは変わらない。マニュアルというより読むための本。
- 山下: UNIX シェルスクリプトコマンドブック第 2 版, ソフトバンククリエイティブ [4 冊]
シェルスクリプトについてだけ説明した本。マニュアルとしても使える。
- スティーブンス: 詳解 UNIX プログラミング, ピアソンエデュケーション [2 冊]
UNIX 特有のプログラミングについての説明。特に並行プログラミングなどは分かりにくいので、本を一回読んで整理すると良い。
- スティーブンス: UNIX ネットワークプログラミング Vol.1 第 2 版, ピアソンエデュケーション [2 冊]
他のコンピュータと通信をするためのプログラミングについての解説。
- 上野: HTTP の教科書, 翔泳社
HTTP プロトコルの詳細についての本。第 5 回, 第 6 回で使う。
- 森: HTML5 & CSS3 ポケットリファレンス, 技術評論社
この授業では HTML についてはほとんど説明しない。最終課題において、ちょっと凝った Web ページを作りたくなったらこの本を使う。

授業の流れと提出物：

1. Linux の使い方 1, 簡単なシェルプログラミング, 並行プログラミング基礎
2. Linux の使い方 2, 並行プログラミング (排他制御と同期) [第 1 回レポート提出]
3. クロス開発, 並行プログラミング (条件変数)
4. メモリマップド IO によるハードウェアの制御 [第 2 回レポート提出]
5. 通信プログラミング: ソケットと HTTP 通信 (前編)
6. HTTP 通信 (後編), 最終課題着手
7. デバッグとテスト, 最終レポートの作成 [インタビュー, 最終レポート提出]

毎回の授業では、授業中に小問題を出題し、これについては授業中に回答を説明する。授業終了近くにやや長い課題を出題するので、次回までに解いてくること。オフィスアワーは木曜の 6 限である。木曜の 6 限は残れるように時間を空けておくこと。

提出の必要があるのは、第2回、第4回のレポートと第7回の最終課題（+ レポート）だけである。ただし、小問題や課題は後で使うので、必ず完成させること。また同じ理由から、作成したプログラムは必ず保存しておくこと。

採点基準:

次のような項目と割合で判断する。

- 1 回目レポート:25%
- 2 回目レポート:25%
- インタビュー: 10%
- 3 回目レポート: 20% + プログラム: 20%

1 第1回

今回の内容

- PC の設定と Linux のブートができるようになる。
- Linux に慣れる。shell スクリプトを使えるようになる。
- C プログラミングの復習。C によるプロセスの生成。
- シグナルによるプロセス間通信。

1.1 Linux マシンの使い方

1.1.1 立ち上げ方

(ネットの設定が終わっていないので) LAN ケーブルは接続せず、電源だけ接続する。ノート PC (以降ではホストと言う) の電源を入れる。しばらくすると、ブートする OS を選択する画面が表示されるが、何もしなければ Linux が立ち上がる。

Linux の種類:

Linux には種類がある。今回の演習のホスト側の Linux は、Debian Linux (リリース名 Stretch, Version 9) である。すべての Linux では OS のカーネル部分は共通だが、カーネルだけでは何もできない。コマンド類等をセットにしたものは distribution と呼ばれる。distribution には Debian, Ubuntu, RedHat, CentOS, Fedora などがある。元々が同じ Linux であり、コマンドなどはかなり共通している。(もちろん、MacOS などの他 UNIX とも似ている。) ただし、管理者として OS の設定をしようとなると distribution の違いは大きい。本を買って勉強する場合、一般利用者用の本はどれを買ってもある程度は使える。一方、管理者用の本はどの distribution 用の本かをよく見て買う。

1.1.2 端末の起動

電源を入れて enshu でログインする (パスワードは授業中に伝える)。「アクティビティ」をクリックする。(または Windows キーを押す。) 検索ウィンドウに「terminal」または「端末」と入力する (日本語入力「半角/全角キー」)。端末アイコンをクリックする。

以降のすべての操作は、この端末の中で行う。なお端末を複数開くには、上のバーの「端末」をクリックして「新しい端末」を選ぶ。

1.1.3 自分のアカウントを作る

自分のアカウントを作るために次を実行する。

```
$ su
# adduser 自分のユーザ名
# gpasswd -a ユーザ名 sudo
# exit
$
```

su のパスワードは口頭で伝える。また、adduser の中でパスワードは必ず入力すること（重要）。その他にいろいろ聞かれるが全部 Enter キーで良い。最後だけ Y とする。これで、自分のアカウントが作成された。（このアカウントを実際に使うのは、もう少し後で。）

Linux においては、システムの管理を行うユーザを特権ユーザと呼ぶ。代表的な特権ユーザは root である。一方、enshu や上記で作成した自分のアカウントは一般ユーザである。上の su は一般ユーザが root になるためのコマンドである。

su のたびに毎回パスワードを入力するのは面倒である。そのために sudo というコマンドもある。sudo の後に任意のコマンドを書くと、そのコマンドは特権モードで実行される。もちろん、誰でも sudo が実行できたら、特権ユーザを分けている意味がない。上の gpasswd で、ユーザを sudo 可能にしている。

以降では、# のプロンプトで始まるコマンドは、su して実行しても良いし、sudo で実行しても良い。

1.1.4 ネットワークの設定確認

LAN ケーブル^{*1}が接続されていなかったら接続し、まずホスト名を確認する。

```
$ hostname
```

(PC 本体表面のマシン名と異なる名前が表示された人だけ以下を実行)

```
# nmcli general hostname pj-linux-～
```

IP アドレスを設定・確認する（LAN ケーブルが接続されていないと確認できないので注意）。

```
$ ip a
```

enp2s0 で始まる行の少し下に、inet で始まる行があり、その中に IP アドレスが表示される（ちなみに "enp2s0" は Ethernet Port 2 Slot 0 の意味）。表示される IP アドレスは次の通りでなければならない。

表1 IP アドレス割当て

マシン名	IP アドレス
pj-linux-0	172.28.34.60
pj-linux-1	172.28.34.61
pj-linux-2	172.28.34.62
...	
pj-linux-15	172.28.34.75
	76～99 は欠番
raspi-0	172.28.34.100
raspi-1	172.28.34.101
...	
raspi-15	172.28.34.115

もし違っていたら、NetworkManager で修正し reboot する（授業中に指示する）。

^{*1} LAN ケーブルのほかに Ether ケーブル, Ethernet ケーブル, 100base-T ケーブルなどの呼び方がある。

```
# nm-connection-editor  
または  
# nmcli c m enp2s0 ipv4.addresses 172.28.34.~/24
```

1.1.5 落とし方

いきなり電源を切ったりしないこと。次のコマンドを実行して終了する。

```
# shutdown -h now
```

画面に一連のメッセージが出た後で、自動的に電源が切れる。少し時間が掛かるので、あせらず待つこと。上記のコマンドの代わりに、GUI で右上のアイコンをクリックしシャットダウンをしてもよい。ただし、休止（こんな感じのアイコン「||」）は押さないこと。立ち上げても画面が黒いまま操作できなくなる。

再度電源を入れて、今度は自分のアカウントでログインする。

1.1.6 個人設定

「端末」はとてもよく使うので、そのための設定をする。先ほどと同様に、「アクティビティ」→検索ウィンドウに「terminal」または「端末」として、端末アイコンを表示する。これを右クリックして、お気に入りに登録する。これで、アクティビティをクリックすると出てくる左サイドバーの中に「端末」が表示されるようになるので、「端末」をクリックすればよい。

後で述べるエディタ gedit を使う人は、同じように検索ウィンドウで「gedit」と入力して、「テキストエディタ」をお気に入りに登録しておく。

パッドのタップによってマウスクリックをしたい場合は、次のようにする。「画面上のメニューバーの右のアイコン（ネットワークやスピーカー）」をクリック→表示されたメニュー中の左下の「スパナとドライバー」のアイコンをクリック→「すべての設定」画面が表示される→「Mouse & Touchpad」をクリック→「タップでクリックする」をオンにする。

ウェブブラウザは、「アクティビティ」を押すと左にサイドバーが出てくるので、その中（たぶん一番上）から Firefox を探してクリックする。

この資料の pdf を USB メモリに入れて、演習用ノート PC にコピーしておく。 USB メモリを挿せばウィンドウがポップアップするので、あとの操作はだいたい分かるだろう。

1.1.7 ファイルの保存について

これから作成するプログラムなどのファイルは、ハードディスクに保存される。ハードディスクは動作中に振動などの衝撃を与えると容易に故障し、ファイルはすべて読めなくなってしまう。自己責任でファイルを必ず保存しておくこと。MyVolume へ保存する（付録 A）か、自分の USB メモリに保存するとよい。（USB は、/media/自分/〜というディレクトリに自動マウントされるので、ここにコピーすればよい。GUI で操作したいときは「アクティビティ」→「ファイル」を選ぶ。）

後で説明する scp を使って、隣の人のコンピュータに保存したりするのも一案である。

1.1.8 「端末」での入力方法

プログラムを停止したい時（無限ループになってしまった場合など）は`^C`を押す。`^Z`も一旦停止するので`^C`に似ているが、使い方が難しいので`^Z`はよく調べて使うこと。

矢印キーの上（↑）を押すと過去に入力したコマンドが表示される。→←を使って移動してコマンドを修正できる。Enter を押すと修正後のコマンドが実行される。（`^R` と押すと過去のコマンド履歴の検索もできる。）

ファイル名を途中まで入力して、TAB を押すとファイル名を自動補間してくれる。

1.1.9 マニュアルの引き方

Linux のコマンドが分からないときは、次の 3 つのコマンドが便利である。

- man: そのキーワードのマニュアルを引く
- whatis: そのキーワードのマニュアル一覧を表示する
- apropos: そのキーワードを含むすべてのマニュアル一覧を表示する

特に man は頻繁に使うので、以下の例を実行すること。man から抜けるには q を入力する。このことをよく忘れるので、必ず一度練習しておく。使用例:

```
$ man ls
```

1.1.10 エディタ

著名なエディタには、以下のようなものがある。

- gedit (GUI で使える一番簡単なエディタ)
- nano (CUI で使える一番簡単なエディタ)
- vi, vim (多機能エディタ。vi と実行すると vim が立ち上がる)
- emacs (多機能エディタ)

gedit や nano は Windows のメモ帳のようなもので、誰でも簡単に使える。本格的なエディットには、vim/vi か emacs を使う (簡単な説明は付録 B)。どちらも有名なエディタで、解説した Web ページなどもたくさんあるので、自分で勉強する。極めて多くのコマンドがあり、使いこなせば作業効率が劇的に向上する。将来、ソフトウェア関係の職業につくつもりであれば、この機会にまともなエディタに慣れておくことを強くお勧めする。

なお、日本語を入力するときは、vi と gedit では「半角/全角」キーを押す。emacs では”Ctrl-\\”を押す。

vi と emacs:

emacs の方が高機能であり、できないことはないというくらい強力なエディタである。vim もかなり色々なことができるし、ほぼすべての UNIX に標準で入っているので利用範囲は広い。

1.2 コマンドの基本

CUI の良いところは、文字だけで指示できることである。過去の指示を保存しておいたり、その指示を少し変形して再利用したりできる。これは CUI でのコマンドをプログラムと見なせるということである。コマンドが入力されたら、それを (プログラムとして) 実行するプログラムを UNIX では shell という。shell には沢山の種類 (sh, bash, csh, tcsh 等) があるが、この演習では bash (多くの Linux でのデフォルトの shell) を使う。

まず、shell の基本的な使い方を説明する。ファイルシステムの基本 (F.1) と shell のコマンドの基本 (F.10) を口頭で説明する。少なくとも以下について理解する。

- ファイルとディレクトリ、絶対パスと相対パス
- ワーキングディレクトリ、ディレクトリの省略記法 (~ . . .)
- リダイレクション (標準入力, 標準出力, 標準エラー出力)
- パイプ
- 最小限のコマンド (cd, pwd, cp, mv, rm, mkdir, rmdir, ls)

小問題 1:

- ls コマンドはファイルの一覧を出力する。ファイル一覧を (画面でなくて) out というファイルに出力せよ。
- cat コマンドは、ファイルの中身を表示するコマンドである。上で作ったファイル out の中身を表示して

確認せよ。

- cat というコマンドは実は /usr/bin というディレクトリにある。これを確認するために、

```
$ ls /usr/bin
```

と実行してみよう。とても沢山のファイルが出力されて、この中に cat があるのかわからない (アルファベット順なので順番に見て行けば分かりますが)。そこで、一旦 ls /usr/bin の結果を out2 というファイルに出力せよ。次に grep コマンドを使って、out2 というファイルの中に cat が含まれるか確認せよ。

- 上の動作をパイプを使って実行せよ (ファイル out2 は作らないという意味)。
- ls out out2 out3 と実行すると、out3 というファイルは存在しないためエラーになる。この ls の実行結果を「エラーも含めて」すべて allout というファイルに出力せよ。

1.3 shell スクリプト

shell の特徴の一つは、自分でコマンドを作れるということである。「隠しファイルを含めて」「色を付けずに」「詳細な」ファイル一覧を表示したいとする。長いコマンドになるので、これを短い l (小文字のエル) というコマンドで実行できるようにしたいとする。まず、次のような内容の l というファイルをエディタで作る。

```
#!/bin/bash
ls -a --color=never -l
```

これを保存し、中身を確認する。

```
$ cat l
```

これを実行するには、次のように入力する。

```
$ ./l
```

しかし、実際にはエラーになって実行できない。これはファイルの属性が「実行可能」になっていないためである。そこで、次のように実行する。

```
$ ls -l
$ chmod u+x l
$ ls -l
```

chmod の u+x の意味は、このファイルの属性を「利用者 (user) が実行可能 (executable)」のように設定せよ、という意味である。u-x なら、利用者が実行不可能という意味である。(これ以外の指定については、自分で調べる。) ls -l を実行すると、ファイルの一覧が属性付きで表示される。では、実行してみよう。

```
$ ./l
```

ただし、いちいち ./l と実行するのではコマンドらしくない。実は、~/bin の下にあるファイルはコマンドとして実行することになっている。このディレクトリを作って、そこに行き、l をそこに置く。

```
$ mkdir bin
$ cd bin
$ mv ../l .
```

ホームに戻って、l というコマンドを実行してみる。

上の l のような shell 用のプログラムはスクリプトとも呼ばれる。shell の各種機能や shell スクリプトについては、付録 F (F.6 など) を参照のこと。

小問題 2:

- 今の時刻 (date コマンド) と今いる場所 (pwd コマンド) と詳細なファイル一覧を表示したいとする。このためのコマンド ll を作りなさい。
- find はファイルを探すコマンドである。例えば,
\$ find . -name ファイル名 -print
などとする。しかし、これは長くて面倒なので、
\$ f ファイル名
とするだけでファイルを探してくれるコマンド f を作りなさい。このためには、引数にアクセスする特殊変数 \$1 を使う (F.7 参照)。

1.4 C プログラミング

1.4.1 復習**たぶんやらない**

C のプログラムを復習するために、次のようなプログラム test.c を作成する。何をするプログラムかは見れば分かると思うが、(C のプログラムがまったく思い出せない人は、/home/enshu/c/の下に幾つかプログラムが置いてあるので、これを見て思い出す。)

```
#include <stdio.h>

int main()
{
    char buff[100];
    scanf("%s", buff);
    printf("%s\n", buff);
}
```

このファイルを test.c に保存して、以下を実行する。

```
$ cc test.c
$ ./a.out
```

cc で C コンパイラを実行する。これにより、C プログラムから実行可能ファイルが生成される。実行可能ファイルは、何も指定しなければ a.out という名前である。ls -l を実行して、確かに a.out というファイルが存在することと、実行可能属性が付与されていることを確認してみよう。

1.4.2 ファイル記述子を使ったプログラム

ここからが新しいプログラムである。ファイルから文字を入力して、それを画面に表示するプログラムを作る。ただし、ここでは fscanf ではなく、ファイル記述子を直接利用する方法で作ってみる。ファイル記述子は OS の入出力の機能を直接使う方法であり、後で作成するプログラムでは多用するため、ここで練習しておく。ファイル記述子を使ってファイルから入力を行うプログラムは、次のようになる。

```
#include <fcntl.h>

int main() {
    char buff[5];
    int fd;
    fd = open("test.txt", O_RDONLY);
    read(fd, buff, 5);
}
```



```

    write(1, buff, 5);
    close(fd);
}

```

ファイルをオープンしてファイル記述子を作る関数は `open` である。ファイル記述子から読み込むには `read` 関数を使う。これによりファイルからデータが `buff` に読み込まれる。

このプログラムでは `fd` がファイル記述子 (file descriptor) である。見ての通り整数 (int) である。ファイル記述子の 0, 1, 2 は特別である。0 は標準入力, 1 は標準出力, 2 は標準エラー出力のためのもので、この 3 つのファイル記述子は最初から用意されている。

```

read(0, buff, 5);

```

などとすると、キーボードからの入力を `buff` に読み込むことができる。

上の `open`, `read`, `write` のように新しいシステムコールやコマンドが出て来たら、`man` コマンドを使って自分で調べる。ただし、コマンド名、システムコール名、ライブラリコール名が重複しているときには、うまく探せない。例えば、システムコールの `open` を調べようとして、`man open` としても、実はコマンドにも `open` があるので、そちらの説明が表示されてしまう。`whatis open` の結果を見ると、次のようになっている。

```

open (1)          - start a program on a new virtual terminal (VT).
open (2)          - open and possibly create a file or device

```

括弧の中が種類を表しており、1 はコマンド、2 はシステムコール、3 はライブラリコールである。

```

$ man 2 open

```

とやると、システムコールの `open` のマニュアルが表示される。

システムコールとライブラリコール:

システムコールとライブラリコールは、C のプログラムから呼び出す関数のことである。システムコールは OS が用意している機能だが、ライブラリコールは OS とは関係なく、C が用意している関数である。ライブラリコールは普通の C で書かれており、自分が書いたプログラムと一緒にリンクして使っている。一方システムコールは、OS の中で定義されている。`printf` や `scanf` はライブラリコールなので普通の関数であり、その中で `write` や `read` システムコールを呼び出している。

プログラムを作る上で両者の違いはあまり意識する必要はないが、前述のように `man` のマニュアルは 2 がシステムコール、3 章はライブラリコールと別れている。

小問題 3:

次の 2 つの修正をして、上のプログラムを `file.c` として完成させなさい。1 つ目の修正は 10 文字を読み込んで表示するようにすることである。2 つ目はエラーチェックをきちんとすることである。`open`, `read`, `write`, `close` を `man` で調べて、そのエラーに対処する。エラーのチェックについては、`perror` という便利な関数がある (G.3 を参照)。

(余力がある人 1) ファイル記述子 2 番に対して `write` を実行し、リダイレクションを使って確かに標準エラー出力に出力が出ていることを確認せよ。

(余力がある人 2) 標準入力 (つまりキーボード=ファイル記述子 0 番) から文字列を入力し、これをファイル (例えば `test.txt`) に書き込むプログラムを作成せよ。さらに `test.txt` がない状態、空のファイルの状態、20 文字くらいの何らかの文字が入った状態で動作させて、`write` システムコールの動作を理解する。

標準エラーへの printf:

printf は実は標準出力に出力している。printf は内部では `fprintf(stdout, ...)`; を実行したことになる。変数 `stdout` には標準出力のストリームが最初から代入されている。標準エラーのストリームは `stderr` という変数に代入されているので、次のようにすれば標準エラーに出力される。

```
fprintf(stderr, "ERROR MESSAGE");
```

1.5 シグナルを使ったプロセス間通信

Linux では、複数のプログラムを同時に動かすことができる。一つ一つのプログラムが動いている状態を「プロセス」と呼ぶ。プロセスは独立した存在であって、互いに干渉することは基本的にはできない。プロセス間でやり取りをするにはプロセス間通信を行う。プロセス間通信には様々な種類があるが、今回は「シグナル」を使う。

シグナルとはソフトウェアだけで行う割り込みのことである。割り込みとはあるプロセスがある処理をしているときに強制的に別の処理を実行させる仕組みである。（「コンピュータアーキテクチャ」の授業では、割り込みはハードウェアが掛けるものであったが、シグナルはソフトウェアが掛ける割り込みである。）

シグナルには種類があり番号が付いている。シグナル番号の一覧は、`/usr/include/bits/sgnnum.h` の 80 行目前後にある。シグナルを受け取る側は、例えば次のように書く^{*2}。

```
void sig_handler(int signum){
    ~
}

int main()
{
    signal(SIGUSR1, sig_handler);
    ~
}
```

`signal` を実行すると、ある番号のシグナル (`SIGUSR1` は既に定義済みの番号なのでこのまま使えばよい) が送られてきた時に実行される関数 (上では `sig_handler`) を登録する。このような関数をシグナルハンドラ（またはハンドラ）と呼ぶ。main の ~ の部分を実行中にシグナルが発生すると、ハンドラが実行される。ハンドラは、1 引数（データ型は `int`）で戻り値は `void` の関数でなければならない。なお、引数はシグナル番号である。

あるプロセスへシグナルを送るには、次のように書く。

```
kill(プロセス番号, シグナル番号);
```

勝手に使ってよいシグナル番号は決まっており、`SIGUSR1` と `SIGUSR2` である。今回は、`SIGUSR1` (実際は 10) を使う。（なお今回は使わないが、自分自身にシグナルを送るときには、`raise` 関数を使う。）

タイマー

もし時間がありそうなら、4.3 節を先取りして説明する。下の課題の `rec.c` の中に、1 秒に 1 回 `"tick\n"` と表示する機能を追加せよ。

課題: `sig.c` と `rec.c` を提出する

上の説明を参考にして、シグナルを送信する側のプログラム `sig.c`、シグナルを受信する側のプログラム `rec.c` を作りなさい。幾つか注意が必要である。2 つの異なる実行可能ファイルを作るには、次のようにする。

^{*2} この書き方は実は古い。今は `sigaction` という関数を使うことが推奨されているが、使い方が大変なので今回はこちらを使う。もちろん、`sigaction` を自分で調べて使っても良い。

```
$ cc -o sig sig.c
```

```
$ cc -o rec rec.c
```

ここで-o は、実行可能ファイルの名前指定である。

rec.c のメインは次のようなプログラムとする。

```
while (1) {  
    printf("z\n");  
    sleep(1);  
}
```

すなわち、1 秒に一回"z"を表示する。

rec を別の端末画面で動かし、もう一方で次のコマンドを実行する。

```
$ ps u
```

PID の欄を見ると rec のプロセス番号が分かる。この番号を sig に与える必要がある。これには 2 つの方法がある。一つは、プロセス番号を scanf で読み込む方法である。もう一つは、次のような形でコマンドのオプションで指定する方法である (1234 はプロセス番号の例)。

```
$ ./sig 1234
```

このようにコマンドオプションの値を使う方法は、G.2 を参照のこと。文字列の数字を整数の数値に直すには atoi が便利である。

sig.c ではシグナルを送った旨を、また rec.c では受け取った旨を表示するだけでよい。

提出は、ノート PC から直接 ScombZ にアクセスするのが簡単である。sig.c と rec.c を ScombZ に別々に (2 回) 提出する。または、USB メモリを使って自分の PC に移してから提出してもよい。(それも難しかったら、sig.c と rec.c を表示して画面を写真撮影して提出する。)

(少し余力がある人)

1. signal が SIG_ERR を返したらエラーなので、これをチェックする (このチェックはデータ型が少し難しいので、分からなかったら聞く)。
2. システムコール getpid を呼び出して、自分のプロセス番号を表示する機能を付ける。(これで ps コマンドで番号を調べる必要がなくなる。)

プロセスを終了させる方法:

fork を使ったプログラムにバグがあると、作成したプロセスが終了しなくなることがある。このような時は、まず ps コマンドを実行してプロセス番号を調べる。そして、次のコマンドでプロセスを殺す。

```
$ kill プロセス番号
```

コマンドでプロセスを作る方法:

これには & (F.5 節) を使う。コマンドの後に&を付けると、そのコマンドは別のプロセスで実行される。これを通称「バックグラウンド実行」と呼ぶ。普通のコマンド入力 (バックグラウンド) で実行するからである。普通のコマンドとバックグラウンドのコマンドは同時に実行されることになる。

PC をもってかえってもよい。ただし自己責任。

2 第2回

今回の内容

- gnuplot コマンド [20 分]
- git コマンド [1 時間]
- プロセスとスレッドによる並行プログラミング [1.5 時間]
- mutex による同期 [1.5 時間]
- 第 1 回提出課題 [1 時間]

2.1 gnuplot

総合プロジェクトが始まると、実験で得られた数値をグラフ化したいことがよくある。Excel を使うのが一つの方法だが、ここではもう一つの方法として gnuplot を紹介する。gnuplot 自身には excel のような計算はできないが、リダイレクションと組み合わせれば強力なツールになる。

gnuplot を実行してみる。gnuplot>というプロンプトが表示されるので、そこに gnuplot のコマンドを入力する。コマンドは沢山あるが、データを表示するには plot コマンドを使う。(付録付録 E も参照せよ。)

plot コマンドを使って、 x^2 と $\sin(x)$ を表示してみよう。(なお、終了は exit コマンド。)

なお上記のように gnuplot に一旦入ってからコマンドを入力するのではなく、gnuplot 起動時に gnuplot のコマンドを指定してしまう方法もある。

```
$ gnuplot -e 'plot sin(x); pause -1'
```

(pause はキー入力を待つコマンド。これを入れないとすぐに終了してしまう。)

/home/enshu/rain.txt を自分のホームディレクトリにコピー (cp) する。ls をしてコピーできたか確認する。このファイルは気象庁が公開している気象データのうち、1900 年～1999 年の各月の降水量と合計である。データは TAB で区切られている。cat で見てそうになっているか確認する。

小問題 1:

rain.txt を用いて 1900 年～1999 年の 1 月の降水量を折れ線グラフで表示せよ。

rain.txt を用いて 1900 年～1999 年の 1 月の降水量と 8 月の降水量を折れ線グラフで表示せよ。

2.2 git の基礎

有名な開発支援ツールである git を使ってみる。git はソースコードのバージョン管理ツールである。バージョン管理ツールを使うと、次のようなことが可能になる。

- 昔のバージョンに戻せる
- 複数人が個別に開発したプログラムを容易にマージできる
- 同時に複数の実装方法を試してみて良い方を採用することが混乱なくできる
- 修正履歴を見てコードがそうになっている理由を調べたりできる

git の用語と基本的な考え方を説明する。(図 1 も参照)

- ワーキングディレクトリ: プログラマが実際に作業するディレクトリ。(そのディレクトリから下のファイルツリー全部が git の管理対象になる。)
- インデックス: リポジトリに登録すべきファイル一覧
- (ローカル) リポジトリ: 過去のすべてのソースコードを格納するデータベースのようなもの。ローカルマ

シン上に置かれる。

- リモートリポジトリ: 遠隔マシンに置かれたリポジトリ。(他の人と共有することが多い。共有しないならリモートリポジトリは必須ではない。)

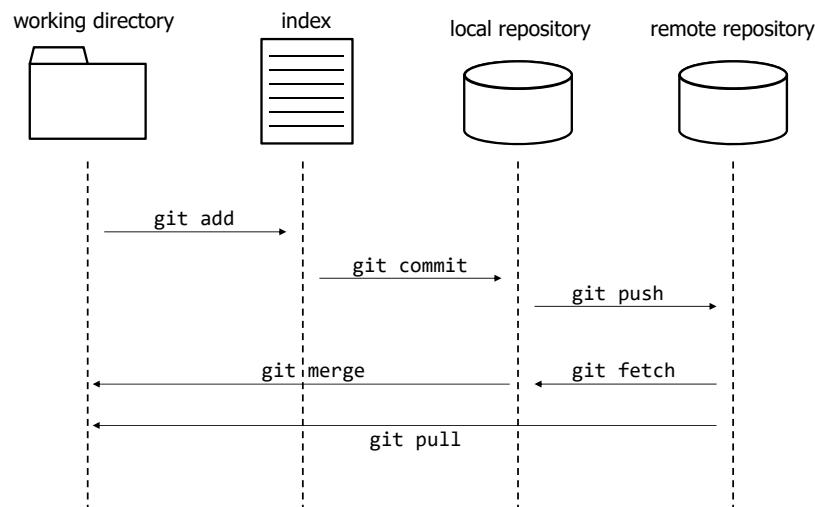


図1 Gitの全体像(「北山: GitLab 実践ガイド, インプレス」を参考に作成)

ホストで以下を実行して、自分の情報を設定する。(実際にこのメールアドレスにメールが送信されることはないが、一応、正しいアドレスを入れること。)

```
$ git config --global user.name "学籍番号"
$ git config --global user.email "メールアドレス"
```

では、実際に git を使ってみよう。

```
$ mkdir proj1          ← ワーキングディレクトリの作成, 名前は自由
$ cd proj1
$ git init              ← ローカルリポジトリの作成
(新たに作ってもよいが) 前に作った C のプログラムを test.c という名前で cp する
$ git add test.c        ← または git add -A
$ git commit -m "コミットメッセージ"
```

その他よく使用するコマンド群。

```
$ git status           ← どのファイルがどの状態にあるのかを確認
$ git log               ← コミット履歴を確認
$ git diff              ← ファイル間, コミット間の差分を表示
$ git reset             ← コミットを取り消す
```

練習として次の操作を行ってみる。

- 先程作成した proj1 ディレクトリで git status の表示内容を確認する
- test.c を一部修正しファイルを保存する。そのあと再度 git status の表示内容を確認する

- `git diff` を実行し、リポジトリの最新内容と今回の変更内容の差分を確認する
- `test.c` を `git add` してインデックスに登録する。そのあと再度 `git status` の表示内容を確認する
- `git commit` で変更をコミットする。そのあと再度 `git status` の表示内容を確認する
- `git log` でコミット履歴として表示される内容を確認する

2.3 GitHub

`git` のリモートリポジトリ (GitHub) を使ってみる。GitHub の設定はあらかじめ伝えたように完了していることを前提に授業を進める。GitHub (<https://github.co.jp/>) とは、Git によるバージョン管理をウェブ上で行えるサービスであり、バージョン管理以外にもチームによるソフトウェア開発を支援する様々な機能を提供している。

Git は分散バージョン管理システムであるため、ローカルリポジトリの他に別マシン上にあるリモートリポジトリに対しても操作することができる。リモートリポジトリを利用することの利点は例えば以下のようなものが考えられる。

- 複数のマシンに同一リポジトリの複製を持つことでデータ損失を防げる
- 製品版 (最新で正しい内容) のファイルをリモートリポジトリで一元的に管理できる
- リモートリポジトリのファイルを壊すことなくローカルリポジトリで色々試行錯誤できる

GitHub をリモートリポジトリとして使用するためには予め GitHub 上にリポジトリを作成しておく必要がある。GitHub 上にリポジトリを作成する方法は付録 D に記載している。

リモートリポジトリを使えるようにするには `git remote add` コマンドを用いる (以降では先程 `git init` したディレクトリ上で操作することを想定している)。

```
$ git remote add origin git@github.com:{username}/{repository}.git
$ git push -u origin master
```

上記の `git remote add` はリモートリポジトリに `origin` という名前を付けている (最初は必ずこの名前にすること)。`{username}` は GitHub で作成した自分のユーザー名で、`{repository}` は GitHub 上で作成したリポジトリ名である (前回作成したローカルリポジトリと同じ名前である `proj1` にすることをおすすめする)。`git push` コマンドでは先に指定した `origin` を使っている。master については後で説明する。オプションの `-u` は、ローカルのブランチとリモートのブランチとを関係付けるための指定である。

次に昔のバージョンに戻す練習をしてみる。上に続けて次を行う。

エディタで `test.c` のどこかを直す

```
$ git status      ← 状態を確認
$ git add test.c
$ git status      ← 状態を確認
$ git commit -m "コミットメッセージ"
$ git status      ← 状態を確認
$ git log
$ git reset --hard HEAD^
$ git log          ← ログが戻ったことを確認
$ cat test.c       ← 元に戻ったことを確認
```

上の `git log` はコミットの記録を表示する。前回変更を行った分と合わせて複数のコミットがあることが分かる。コミットを指定するには幾つかの方法がある。`git log` の結果を見ると、各コミットにハッシュ値が付いている。この先頭の数文字を入れることで、そのコミットを指定できる。省略記法を使うこともできる。HEAD は一番最新のコミット、HEAD^ はその一つ前のコミットである。上の `git reset` は、一つ前のコミットに戻している。実際にファイルが元に戻っているか `cat` で確かめてみる。(上記の `git reset` の引数の説明も含めて、`git` の幾つ

かの便利なコマンドの説明が付録 C にある.)

(時間があれば) 上記は, リモートに push していなかった場合の戻し方である. リモートに push してしまったものを取り消すことは, 基本的にはできない. 別の人がそれを利用して, 別の修正をしているかもしれないからである. そのような場合は, reset の代わりに revert を使う.

修正して, add, commit, push をする.

```
$ git revert HEAD
$ git push -u origin master
$ git log
```

revert を実行すると, エディタ画面になるのでコミットメッセージを確認してから ^X を押す. revert はコミットを取り消すのではなく, 取り消しに相当する内容の新しいコミットを作成する. 上の例は, HEAD を取り消すようなコミットをせよという意味になる. 上で push したときに, もし別の人が修正していた場合はエラーになるので, 後で説明する merge をする.

2.4 プロセスによる並行プログラミング

前回はプログラムを実行するとプロセスができると説明したが, 実際は fork という関数を呼ぶとプロセスが作成される.

```
pid = fork();
```

fork 関数を呼ぶと, 今実行中のプロセスをコピーした新しいプロセスが作られる. 元から実行していたプロセスを親, 新しく作られたプロセスを子と呼ぶ. 子は親のコピーなので, 親と子はまったく同じプログラムであるが, fork の戻り値だけが異なる. 親には子供のプロセス番号が返され, 子には 0 が返される. これを使うと, 自分が親なのか子なのかを区別できる.

プロセスをコピーすると変数もすべてコピーされる. 親と子の変数は, プロセスの丸ごとコピーにより同じ値になるが, 変数としては別である. 従って, どちらかが変数の値を変えても, もう一方にはその値は伝わらない.

なお fork を man で調べると, プロセス番号 (上の pid) は pid_t というデータ型だが, これは実際は int である.

小問題 2:

fork して次のことをするプログラム fork.c を作成せよ. fork のエラーもチェックすること.

- 子は, "child" と表示しては 2 秒スリープすることを 10 回繰り返す.
- 親は, "parent" と表示しては 1 秒スリープすることを 10 回繰り返す.

(余力がある人) 次のようなプログラムを作成せよ. 子ごとに違う値とする部分がポイントである.

- 親は, n 個の子プロセスを作る (とりあえずは n=2 でもよい).
- n 番目の子は, n の値を表示しては 1 秒スリープすることを 10 回繰り返す.
- 親は子プロセスを n 個作った後, "parent" と表示しては 1 秒スリープすることを 10 回繰り返す.

2.5 スレッドによる並行プログラミング

スレッドについて説明する.

- プロセスの問題とスレッドの特徴
- スレッドの生成と終了

スレッドの生成方法と終了方法について、授業形式で説明する。pthread_join は必ず実行しなければならないが、pthread_detach を実行する方法もある（興味があれば自分で調べる）。

なお、スレッド用のライブラリを利用するにはコンパイル時に次のオプションが必要である。

```
$ cc -pthread ファイル名
```

小問題 3:

次のようなスレッドを使ったプログラム thread.c を作成する。親スレッドは、parent と表示し 1 秒の sleep を 10 回繰り返す。子スレッドは、child と表示し 2 秒の sleep を 5 回繰り返す。■

スレッドは幾つでも作ることができるが、CPU は一つ（ないしは数個）しかない。スレッドは実際には同時に実行されているのではなく、少しずつ細切れに実行されている。以下では、これを実際に目で見えて確認することで、スレッドの動きを体感してみる。

スレッドが実際に動いた証拠を取るために現在時刻を測定し、そのデータを収集する。これをグラフにプロットすると、プログラムが動いている時間帯と動いていない時間帯を見ることができる。

clock_gettime() 関数は、高精度の時間を扱うカーネルコールである。使い方は次の通り。

```
clock_gettime(CLOCK_REALTIME, &x);
```

これにより、x.tv_sec と x.tv_nsec に現在の時刻情報を得られる。これを使って、次のようなプログラムを作成する。まず、グローバル変数の配列として次を用意する。

```
#define COUNT 1000
#define LOOP 10000
struct timespec ts[5][COUNT];
```

main においては、まず clock_gettime で現在時刻を測定する。次に（main 以外に）スレッドを 5 つ作成する。各スレッドでは次を実行する。thn はスレッド毎に異なる番号で 0~4 とする。（スレッド毎に異なる番号とするためには、pthread_create の引数を使ってスレッドに値を渡す必要がある。）

```
for (i=0; i<COUNT; i++) {
    busy();
    clock_gettime(CLOCK_REALTIME, &ts[thn][i]);
}
```

ここで関数 busy は、for で LOOP 回ループするだけの関数を自分で作る。これにより、配列 ts[thn][~] に時刻が COUNT 個だけ入る。

main はスレッドを 5 つ作ったら、全スレッドの終了を待つ。そして次のような形式になるように配列 ts の各値を出力する（経過時間とスレッド番号の間の区切りは TAB）。

```
経過時間   スレッド番号
```

ここで経過時間の単位はナノ秒とし、main で最初に測定した時刻を起点（0 秒）とする。（注意: ナノ秒を単位とすると、2 秒程度で int の限界となる。このため、この計算では int でなく long を使うこと。long は 8 バイトなのでナノ秒でも十分に大きな数を表現できる。）

上記の出力をファイルにリダイレクションし、それを gnuplot で plot してみる。このプログラムでは、5 つのスレッドが同時に実行される（main を含めると 6 つだが main は待っているだけで計算はしない）が、利用しているノート PC には、CPU コアは 4 つしかないので、5 つのスレッドを真に同時に実行することはできない（lscpu コマンドで CPU の数を調べられる）。このため、gnuplot の表示の各スレッドに対応した線には、時々隙間ができるはずである。

CPU コアの数の効果をもっとはっきりと見るには `taskset` を使う。

```
$ taskset -c 0-1 コマンド
```

これにより指定したコマンドが 0 と 1 の 2 つのコアだけで実行される。"-c 0-2" とすれば 3 つのコアとなる。(このコマンドは、特定のプロセス・スレッドが動作する CPU を固定して実行性能を向上させるために本来は使う。)

小問題 4:

上記のプログラムを `5threads.c` として作成せよ。 `taskset` でコア数を減らして実験してみる。

(余力のある人) `sched_yield` というシステムコールがある。これを調べて実際に使ってみる。例えば、上記のプログラムで現在時刻を測定した直後にこの関数を呼ぶと、動作はどのように変わるだろうか。

(さらに余力のある人) `sched_setscheduler` というシステムコールがあるので、これを調べて使ってみる。ある一つのスレッドにだけ `SCHED_IDLE` を指定すると、`gnuplot` 表示が分かりやすく変化する。また、`SCHED_FIFO` も分かりやすいだろう。

2.6 様々な同期手法

並行プログラミングにおける同期とその詳細については、授業形式で説明する。以下についてパワポの資料で説明する。

- 共有リソースへのアクセス競合
- 同期処理とクリティカルセクション
- 相互排他 (mutex)
- スターベーションとデッドロック

小問題 5:

`mutex` を使うと確かに一つのスレッドしか実行されないことを、目で見えて確認しよう。このために上のプログラムの各スレッドが 1 つの `mutex` を取りあうようにし、`gnuplot` で確認する。`mutex` を取って処理 A (これがクリティカルセクション) をし、`unlock` して処理 B をすることを 100 回ほど繰り返す。処理 A としては、`busy()` して時刻測定することを 10 回ほど繰り返す。(10 回や 100 回という値は `gnuplot` の結果を見ながら適当に調整する。配列 `ts` をはみ出さないように注意。) 処理 B としては、`busy()` を 1 回だけ実行する。(処理 B がないと `unlock` してすぐ `mutex` を取るため、`gnuplot` の表示が分かりにくくなってしまう。)

また、スレッドの数は 1 つ減らして 4 つとする。4 コアで 4 スレッドとすれば (ほぼ) すべてのスレッドが走るので、`gnuplot` の表示結果がより理解しやすくなる。(実際は他のプロセスも裏で実行されるので、それほど単純ではない。)

第 1 回提出課題

`mutex` は 1 個のリソースでありリソースを得たものだけが実行できる、と考えることができる。講義で説明したように `trylock` を使うと 2 つの `mutex` のうちいずれかが `lock` できた時だけ実行するようにできる。これは 2 つのリソースを複数スレッドが取りあう実行と考えることができる。以下のような有限個のリソースを取りあうマルチスレッドプログラムを作成せよ。

- 問題 1: 上の小問題 5 をベースとして、以下のいずれか (できれば 1C) のプログラムを作成し、その動作状況を `gnuplot` を使った方法で表示・確認しなさい。プログラムには適当なコメントをつけること。
(プログラム 1A) どうしてもできない時は、上の小問題をそのまま提出する。

プログラム 1B (上に述べた) 4つのスレッドが2つのリソースを取りあうプログラムを作成しなさい。

ヒント: mutex を2つ用意し, trylock でどちらか取れる方を取得する。取得できたら, もう一方の mutex も trylock する。もし2つ目のロックに失敗したら1つ目のロックを解放して最初からやり直す。

プログラム 1C 4つのスレッドがあり, 各スレッドは3つのリソースから2つを取ったら実行可能とする。そのようなプログラムを作成しなさい。ヒント: mutex を3つ用意し3個のうち2個が取れるまで trylock すれば良いが注意点がある。1つ目が取れて2つ目が取れなかったら, 1つ目を unlock して0個の状態からやり直すという点である。(これは結局1つのスレッドしか実行しないので, 見掛けは面白くない。スレッドによって必要なリソース数を変えると面白いが, 作るのはちょっと大変。)

- 問題2: 上のプログラム 1B は trylock を使っているのでデッドロックにはならないが, きちんと書かないとライブロックになることがある。ライブロックになる時の動作を説明しなさい。
- 問題3: 上のプログラム 1C において, ヒントに書かれたような動作にしないとデッドロックになる。デッドロックになる時の動作を説明しなさい。

注1: プログラム 1A/1B の場合も, 問題2と問題3は必ず回答すること。

注2: 提出にあたり画面キャプチャも付けること。「PrntScr」キーを押すと, 「画像」ディレクトリの中に png ファイルの形式でキャプチャデータが作成される。うまく行かなかったら, 画面を写真撮影したものでもよい。

スレッドの終了とシグナル:

スレッド内で `pthread_exit` を実行すると, そのスレッドは終了する。あるスレッドを外から強制的に終了させるには, `pthread_cancel` を使う。`pthread_cancel` は内部でスレッドにシグナルを送っている。ただし, `pthread_setcanceltype` で設定が必要など, 使い方がややこしい。これに限らず, シグナルとスレッドはそもそも混在して使うと動作が定められていない場合などがあり, 注意が必要である。

スレッドセーフとは?

man を見ていると, 時々, 「～はスレッドセーフではない。」という表現がある。スレッドから同時に呼んでも問題ない関数(システムコール, ライブラリコール)はスレッドセーフと呼ばれる。スレッドセーフでない関数を複数のスレッドから同時に呼ぶと予想外の動作となる。例えば, `rand` や `strtok` などはスレッドセーフではない。man 7 `threads` に一覧がある。

なお, これに関連してシグナルハンドラの中で呼んでも問題ない関数も定義されている。man 7 `signal-safety` に一覧がある。

3 第3回

今回の内容

- 前回の課題の説明
- git の続き
- クロス開発とは?
- ターゲット (Raspberry Pi) の立ち上げ
- 並行プログラミングの同期 (前回の続き)

3.1 Git の 3 回目: ブランチ

git log を見て分かるように、過去のコミットは時間軸上に並んでいる。これが一連の修正の流れである。例えば、軽微な修正を引き続き行いつつ、(失敗するかもしれない) 大きな修正を同時に行いたいとする。git では修正の流れを枝分けすることができる。これをブランチと呼ぶ。

ブランチには名前を付ける必要がある。実は元々のブランチには master という名前が自動的に付いている (なお最近の git ではデフォルトの名称が master から main になっている)。例えば develop という名前の新しいブランチを作るには、次のようにする。

```
$ git branch develop      ← ブランチの作成
```

また、ブランチを切り替えるには、次のようにする。

```
$ git branch              ← 今いるブランチの確認
$ git checkout develop    ← ブランチを develop に切り替える
$ git branch              ← 今いるブランチの確認
```

develop ブランチの中で、今までの git コマンドを使ってバージョン管理をしながら修正を行う。大修正が無事成功し、元のプログラムにこの修正を反映したいとする。しかし、元のプログラムも微修正されている (はずだ) から、大修正プログラムをそのまま元のプログラムに上書きするわけにもいかない。このような場合に、2 つのブランチを統合してくれるのが git merge コマンドである。以下は develop ブランチで行った修正を master ブランチに取り込んで統合する例である。

```
$ git checkout master      ← 取り込む元のブランチ (master) に切り替える
$ git merge develop        ← もしエディタが起動されたらメッセージを入力する
```

統合と言っても別々のファイルに対する修正は、そのまま修正後のファイルを使うだけである。一つのファイルを master ブランチ (微修正側) でも develop ブランチ (大修正側) でもいじってしまった時は、もしそれらの修正が関係ない (離れた) 箇所への修正であれば、merge は修正を自動的に統合してくれる。ただし同じファイルの同じ行を両側で修正すると、手動で直すしかない。次の小問題でこれを実際に体験してみる。

小問題 1:

develop ブランチを作成する。master ブランチで test.c を修正しコミットする (コミットして初めてブランチができる)。次に develop ブランチに行き、異なる修正を test.c に行いコミットする。(この時の修正は master の修正と同じ行にすること。)そして、develop を master にマージせよ。test.c に対しては CONFLICT という警告が出るはずなので、手で修正して、これをコミットする。最後に git log --graph でログをグラフ表示して、枝分かれが起きてまたそれが統合されていることを確認せよ。

(余力のある人) 今回は実際に複数人で開発しているわけではないので、リモートリポジトリのファイルが知らない間に書き換わることはないが、他の人と開発していればこれはよくあることである。これを模擬するため、次のようにしてリモートリポジトリを書き換えてみる。

GitHub の Web ページ中の真ん中上部のあたりにある「Add file」をクリックするとメニューが出てくる。「Create new file」を選ぶと、新規ファイル作成画面になる。ファイル名と適当な内容を入力し、画面下部にある「commit」をクリックする。

その後に以下のコマンドを実行する。

```
$ git fetch origin master    ← リモートブランチを取得
$ git merge origin/master    ← ローカルブランチにマージ
```

1 つ目のコマンドでは、まず origin の master ブランチをローカルに取得する。これには origin/master という名

前が付けられるので、これを merge する。(なお上の 2 つのコマンドは `git pull origin master` という一つのコマンドの実行と同じである。) merge が完了したら push で戻す。

一人で開発する状況でも、リモートリポジトリはバックアップとして使える。また、あちこちのマシンで開発するときにも便利である。ローカルリポジトリもワーキングディレクトリもない状態でリモートからリポジトリをコピーするには、次のように `git clone` コマンドを実行する。(git clone は、リモートリポジトリをローカルにコピーするコマンドである。)

```
$ git clone git@github.com:{username}/{repository}.git
```

上の URL は、Web のプロジェクト画面でも確認できる。あとは、各々のマシンで必ず pull → 開発 → push という手順で作業すれば、マシン間でバージョンが食い違うことは無くなる。

3.2 コンピュータ間のファイル転送

scp の仕組みを説明する前に、まず**サーバー**と**クライアント**の定義を口頭で説明する。scp のための scp サーバーが各ホストで動作している。scp コマンドを実行する側が scp クライアントになる。クライアントとサーバーの関係と、ファイルを転送する方向とは何の関係もないので注意。

コンピュータ A から B にファイルを転送するには、scp コマンドを使う。scp コマンドは次のように書く。

```
$ scp コピー元 コピー先
```

意味としては cp と同じである。ただし、コピー元かコピー先のいずれかに別ホストのファイルを指定できる。このときは、ホスト名:ファイル名と書く。さらに次のように書けば、あるホストのあるユーザのあるファイルという指定もできる。

```
ユーザ名@ホスト名:ファイル
```

scp クライアントを実行すると、別ホストの scp サーバーにログインする必要がある。このため、scp はパスワードを聞いてくる。どのホストの scp サーバーに何という名前でログインしようとしているのか考えて、適切なパスワードを入力する。

実際に本日の演習に必要なファイルをノート PC に転送する。ノート PC を scp クライアントとして動作させ、172.28.34.60 という scp サーバーから、pj.tgz というファイルを転送せよ。アカウントは enshu を使う (パスワードは既に知っているはず)。

取得したファイルは、沢山のファイルを一つにまとめたものなので、次のコマンドで解凍する。

```
$ tar xvf pj.tgz
```

これによって、pj というディレクトリが作成される。

3.3 クロス開発とターゲットの使用方法

まずクロス開発について説明する。

- ターゲットとは
- ホストとは
- クロスコンパイルとは何か? なぜ必要か?

3.3.1 Raspberry Pi のブートの仕組み

一般の PC は、HDD (Hard Disk Drive) に入っているブートプログラムを読み込み、それによってブートする。(なお、何も入っていない HDD に最初にブートプログラムや OS をインストールする時は、CD-ROM/DVD-ROM などから一旦立ち上げて HDD に書き込む。) 組み込みシステムでは、HDD のような取り外しできない記録装置でなく、取り外し可能なメディアを使ってブートすることが多い。これはホスト側で OS などの開発をするためである。ホスト側でメディアにブートプログラムや OS を書き込み、それをターゲットに装着してブートする。

Raspberry Pi の場合、SD カード、USB 接続の HDD、ネットワークブートが可能である。本演習では SD カードからブートする。

3.3.2 ターゲットとのシリアル接続および起動

Raspberry Pi にはキーボードも画面も接続されていない。USB と HDMI を使えばそれらを接続することはできるが、組み込み機器としてはそのような使い方は一般的ではなく、シリアル接続 (本当は RS-232C と言う) で利用することが多い。本授業でも (練習のため) シリアル接続を用いる。シリアルケーブルが接続されていない場合は、自分でコネクタを差し込む。Raspberry Pi 正面向かって右の UART と書いたピンにシリアルケーブルの 3P コネクタを挿す。この時、上から「白緑黒」とすること。なお、このコネクタはさしっぱなしで良い。(このタイプのコネクタは挿抜耐久性が低い。)

ホスト側では、ホストのキーボード (と文字表示) をシリアルに入力するソフトを動かす必要がある。これが minicom というソフトである。以下のように実行する。

```
# minicom
```

minicom を動作させたら、SD カードを挿し、LAN ケーブルを (ホストと) 接続する。そして Raspberry Pi を起動する。

電源を接続すると、minicom 画面に Linux がブートする過程のメッセージが表示されるはずである。立ち上がったら、pi でログインする (パスワードは授業中に伝える)。

以下を入力して IP アドレスを確認する (eth0 という行の下 inet の所)。

```
$ ip a
```

IP アドレスが表示されない、または表示されたが間違っているなら、教員または TA に伝える。

3.3.3 ターゲットの終了

ターゲットも、いきなり電源を切ってはならない。電源を切るには、ホストと同様に shutdown コマンドを実行する。reboot: Power down というメッセージが表示される。その後、Raspberry Pi 本体 (側面) にある 2 つの LED のうち左側の LED が点滅する。LED が完全に消えたら、電源アダプタを抜いてよい。(まれに Reached target Shutdown まで出て、上のメッセージが出ないことがある。原因は不明。それでも電源を抜いて構わない。)

minicom の終了は、^A, z, q, (Yes か聞かれるので) Enter と順番に入力する。

3.4 クロスコンパイル

簡単なプログラムを作り、これをターゲット用にクロスコンパイルし、ターゲットに転送して実行するという一連の手順を実行してみる。ホストとターゲット間の転送は、やはり scp を使う。ただし、**ターゲットは scp サーバにはなれない**ことに注意。

既に ARM 用クロスコンパイラ arm-linux-gnueabi-hf-gcc はインストール済みである。クロスコンパイルしたいファイルが main.c だとすると、

```
$ arm-linux-gnueabi-hf-gcc main.c
```

とやってみる。a.out ができたはずである。

```
$ file a.out
```

と実行すると、ARM 用のコードが出来ていることが分かる。(普通の cc でコンパイルした a.out にも file をして、その結果を比較してみるとよい。)

小問題 2:

1~10 億までの数を加えるプログラムを作り、main.c とする。これをまずホストで実行して、実行時間を測定する。(実行時間を測定するには time コマンドを使う。) 次に、同じファイルをクロスコンパイルしてターゲットで実行せよ。ターゲットでの実行時間はどの程度になったか。

(また、cc でコンパイルした実行可能ファイルをターゲットで実行すると、何が起きるか。)

(余力がある人) time コマンドの real 時間は、前に使った clock_gettime で測定できる。time コマンドの user 時間は、clock_gettime の第 1 引数を変更すると測定できる。time コマンドでなく、プログラムの中で user 時間を測定するプログラムを作成せよ。

3.5 scp を便利にする

scp のパスワードを入れる方法の場合、毎回ネットワーク上をパスワードが送信される。送信は暗号化されているものの、パスワードが解読される可能性はある。また、パスワードを毎回入れるのも面倒である。公開鍵を使えばこれらの問題を解決できる。

scp クライアント側において次を実行する (ed25519 は楕円曲線暗号方式を指定している)。

```
$ ssh-keygen -t ed25519
```

この時、passphrase では単に Enter キーを押すこと。これにより、~/.ssh の下に id_ed25519 と id_ed25519.pub という 2 つの鍵ファイルができる。前者が秘密鍵、後者が scp サーバに置くための公開鍵である。scp を使って、id_ed25519.pub を scp サーバ上の ~/.ssh/authorized_keys というファイルにコピーする。(このときファイル属性を"go-r"にする必要があるかもしれない。)

クライアントで scp を実行してみると、パスワードが聞かれなくなっているはずである。

(なお、ssh-keygen で passphrase を付けないと、秘密鍵が盗まれた時に自由に利用されてしまう。passphrase を付け、かつ ssh-agent を使うのが安全性も利便性も高い方法である。ここでは若干簡易的な利用をしている。)

3.6 条件変数による同期

前回の課題では trylock を使い、lock できない時は繰り返し trylock を行った (このように繰り返しで条件が成立するまで待つことを busy wait と言う)。このような繰り返しは CPU の無駄使いである。その条件が成立した時に別スレッドから教えて貰う仕組みがあれば、ずっと効率が良い。このための仕組みが条件変数である。

■ 条件変数とは何か?

小問題 3:

ある銀行内に多くの口座があり口座間で頻繁に振替が行われているとする。一つの銀行の中での振替だけでなく、全口座の総額は変わらないことに注意。一回の振替を実行することをトランザクションと言う。大量のトランザクションがあるため複数のスレッドで処理する。振替の内容は ~/pj/txn/trans.csv に入っており、次のようなデータが (文字で) 入っている。

t,1234,5678,1000

t,2143,6587,2000

...

最初の行は 1234 番口座から 5678 番口座へ 1000 円を振替える、という意味である。t は普通のトランザクションであることを表すが、とりあえずは必ず t だと思ってよい。trans.csv が何行あるかは、wc コマンドを使って調べる (10000 行のはずである)。これを N 行だったとする。

次のようなプログラムを作成しなさい。まず、上記の trans.csv を読み込み、from[N], to[N], amount[N] という配列に読み込む。(最初の t は捨ててしまってもよい。) また各口座を表す整数の配列 account (口座数は 100 とする) を作成する。そして各口座の初期値を 10000 円とする。(額がしょぼいと言って勝手に増やさないこと。必ずこの金額にする。) スレッドを 10 個作成する。from/to/amount の配列を 10 分割し、各スレッドは N/10 個のトランザクションを順に実行する。この時、あまりに実行が早いので、スレッドが十分な期間重なって並行実行されない可能性がある。このため、今回は実際の振替処理をする前に、**振替金額 × 100 回程度の何もしない for ループ**を挟むこと。なお、main ではもちろん pthread_join を実行する。

全口座の総額を計算する関数 sumAmount を作成する。全スレッド実行前と全スレッド完了後に sumAmount を呼び、それらが一致しているかどうかを表示して確認する。(上記の通りに作れば一致しない可能性が高い。) また前回課題を参考にして、スレッド実行前と実行後の間の時間を測定する。

ストリームからの入出力 (「プログラミング 1・2」で十分やったはずなので簡単に復習):

上記の trans.csv からの読み込みはストリームを使う方が簡単である。

```
FILE *istream;
istream = fopen(~, "r");
while (1) {
    val = fscanf(istream, "%c,%d,%d,%d\n", ~);
    if (val==~) break;
    ~
}
```

```
fclose(istream);
```

fscanf の第 2 引数は入力される形そのままに書くのがミソである。なお、~ の部分は自分で考え、エラー処理も自分で書くこと。

課題: transaction.c を提出する

上記のプログラムには 2 つ問題がある。それを解決するため次の機能を追加したプログラムを作りなさい。ファイル名は transaction.c とする。まず sumAmount が一致しないのは大問題なので、全体で mutex を 1 つだけ作り、トランザクションの開始前に lock し、終了後に unlock をする。(トランザクションとは振替一つずつのことであり、スレッドの中の繰り返し全体のことでない。)

もう一つの問題は口座残高が 0 円以下でも動いてしまうことである。そこで条件変数 cvar を 1 つ作成する。トランザクションの中で from の口座が amount 円未満ならば cvar で cond_wait する。一方、口座に振替をしたら cvar に cond_signal を送り、待っているスレッドを起こすようにする。この方法だと、関係ない口座に振替をしてもスレッドが起こされてしまうので、起こされた側ではもう一度金額を確認し、まだ金額が少なければまた cvar で待つようにする。

余力問題 1: 上記のようにスレッドが cond_wait で停止するとデッドロックする可能性がある。簡便な回避方法として、pthread_cond_timedwait を使う。一定時間待っても待ちが解消されなかった場合は、そのトランザクションは実行せずにそこから下のトランザクションを先に実行する。全部実行し終わってから再度実行すればよい。(そこまでやっても残高不足になる可能性はある。それは単なる赤字なのでやむを得ない。)

余力問題 2: 条件変数を口座毎に作って、その口座への振替を待っているスレッドだけを起こすようにする。た

だし、cond_wait は cond_signal 以外でも起こされることがあるため、起きた後に口座にお金が入っているかを必ず確認しなければならない (transaction.c は元々そうになっているはずである)。

printf とバッファリング:

printf は、(既に述べた通り) stdout というストリーム (FILE *) に対して出力をしている。ストリームに出力すると、OS は出力回数をできるだけ減らそうとする。つまり、出力した文字を OS の中にギリギリまで溜めこむ (これをバッファリングという)。これにより、printf した文字がすぐ画面に表示されなくなる (ことがある)。しかし、微妙なタイミングを調べたい時は、直ちに文字を表示したいことが多い。一つの方法は、改行を付けることである。多くの場合はこれで改善されるが、それでも必ず表示されるとは限らない。

動かしてみてタイミングがおかしいなと感じたら、次のようにすると良い。

```
printf(~);
tcdrain(1);
```

ここで tcdrain は、バッファリングされた文字が端末に表示されるまで待つ関数である。

4 第4回

今回の内容

- make
- mmap とは何か?
- メモリマップ IO によるハードの制御
- レポート出題

4.1 分割コンパイル

これまでプログラムは一つのファイルだった。大きなプログラムを手分けして開発する場合などは一つのプログラムを分けて記述したい。これが分割コンパイルである。分割コンパイルをするには、以下のことに注意して複数のファイルに分ける。

- 関数毎にまとめる (一つの関数の途中で分けることはできない)
- #include はそれぞれのファイルの先頭を書く
- 関数のプロトタイプ宣言を (使用する) すべてのファイルに書く
- グローバル変数も (使用する) すべてのファイルで宣言する
- main 関数は全体で一つだけ

コンパイルは次のように行う。

```
$ cc ファイル1.c ファイル2.c ファイル3.c ...
```

~/pj/maketest の下に f.c, g.c, m.c というファイルがあるので、プログラムの内容を確認する。

```
$ cd ~/pj/maketest
$ cat f.c
$ cat g.c
$ cat m.c
```


3つのファイルをまとめてコンパイルし、実行せよ。

4.2 make

大きなプログラムは、膨大な数のソースファイルから構成される。その中の一部を修正した時に、何をコンパイルすべきかを人手で管理するのは大変だし間違いやすい。make はこれを自動化してくれる。

■ make の説明

次のような内容のファイル Makefile を作成する。

```
m: f.o g.o m.o
f.o: f.c
g.o: g.c
m.o: m.c
```

Makefile とは、コマンド **make** が参照するファイルで、実行可能ファイルの作り方が書いてある。上の1行目は、「m は f.o, g.o, m.o に依存している」という意味であり、「m は f.o と g.o と m.o から作れる」という意味でもある。o というファイルは、.c という C プログラムから作られる中間ファイルである。次の行は、f.o は f.c に依存しているという意味である。（以降はコロンの左をターゲット、右は依存ファイルと呼ぶ。）

実験: まず **make** を実行して、必要なファイルを全部コンパイルして、m を作ってくれることを確認する。次に、エディタを使って f.c をエディットする（スペースを足す、あるいはメッセージを書き換えるなど）。もう一度 **make** を実行して、何がコンパイルされるかを確認する。その他、m.c をエディットしたり、m や f.o を削除したりして、その都度 **make** を実行してみる。これにより、必要最小限の動作しかしないことを確認する。

上では依存関係を書いただけであり、まだ **make** の半分の機能しか使っていない。ある依存関係があった時に、どのようにターゲットを作ればいいのかを書くことができる。Makefile を次のように修正する。

```
m: f.o g.o m.o
    cc -o m f.o g.o m.o
f.o: f.c
    cc -c f.c
g.o: g.c
    cc -c g.c
m.o: m.c
    cc -c m.c
```

ここで、cc の左はスペースではなく、TAB である（重要）。TAB で始まる行は、その上の行の依存関係が満たされなかったら、TAB の右を実行せよという意味である。最初に実験したように、TAB で始まる行がなくても、make は適当に作ってくれる。しかし、特殊なことをする場合は、作成方法を TAB の行で指定しなければならない。

また、デフォルトのターゲットは一行目の左側であるが、**make** にオプションを付けるとターゲットを指定することができる。

小問題 1:

Makefile が正しくないと、必要なファイルを消してしまうなどの可能性がある。まず **make -n** を実行すると、実行されるはずのコマンドが表示だけされる。これで確認した後に **make** を実行する。

1. クロスコンパイルするための Makefile に修正せよ。

2. `make clean` と実行したら、いつも `m` と `*.o` を削除するように Makefile を修正せよ。(ターゲット、依存ファイル、実行コマンドに何を指定するかを考える。)
3. (余力のある人) `make all` と実行したら、第 1 回の `sig.c` と `rec.c` から `sig` と `rec` を作る Makefile を作れ。下の説明を参照せよ。この Makefile は `sig.c` と `rec.c` が存在する場所に作る。

`make` を使って、2つのファイルをコンパイルすることもできる。このためには、Makefile の 1 行目を次のように書く。

```
target: sig rec
```

実際にはターゲットの名前は何でもよい。`make` はそのターゲットを作成しようとする。そのためには、`sig` と `rec` が必要だというのが、このルールの意味である。しかし、`target` の作り方が 2 行目に書いてないので、永久に `target` は作られない。つまり、`make` は実行されるたびに、`sig` と `rec` を作るしかないのである。(これとは別の方法として、`make sig rec` とオプションで 2 つ指定してもよい。)

4.3 タイマー

時間が経過したことを知りたいことがある。`sleep` を使えば待つことはできるが、その間、他の処理ができなくなる。タイマーの機能を使うと時間経過した時にシグナルを送ってくれる。タイマーを仕掛けるには次のようにする。

```
struct itimerval timval;
timval.it_interval.tv_sec=1;
timval.it_interval.tv_usec=0;
timval.it_value.tv_sec=1;
timval.it_value.tv_usec=0;
setitimer(ITIMER_REAL, &timval, NULL);    // タイマー開始
~ // 別の仕事
```

`setitimer` の実行から 1 秒経過した後に `SIGALRM` というシグナルが発生する。(参考までに `SIGALRM` は 14 である。)あとはシグナルハンドラを書けばよい。なお上の書き方だと、その後も定期的に 1 秒ずつシグナルが発生する。1 回だけのタイマーとしたり 1 秒という時間指定を変えたりする方法は、自分で調べること。

小問題 2:

`main` は 1 秒に 1 回 `"z"` と表示し、同時にタイマーを使って 1 秒に 1 回 `"hello"` と表示するプログラム `timer.c` を作成せよ。第 1 回に作った `rec.c` をベースにして次の変更をする。まず、シグナル番号は `SIGUSR1` ではなくタイマー用の番号を使うように変更する。そして、その次に 1 秒間隔 (`interval`) で継続的に起動されるタイマーを仕掛けるようにコードを追加する。

それが完成したら、1 秒後に 1 回だけ起動されるタイマーに変更する (提出課題で使うため)。

4.4 デバイスの操作 - I2C デバイス

ここでは Raspberry Pi の I2C に接続されたデバイスを `C` を使って操作する。I2C (正式には I^2C) は 3 線だけでデバイスを安価に接続できるバスで、センサーなどのデータ転送量の少ないデバイス接続に用いられる。Raspberry Pi では I2C に液晶ディスプレイ (LCD) と温湿度/気圧センサーが接続されている。

I2C はマスター (PC) と複数のスレーブから構成される。スレーブにはアドレスが割り振られている。I2C で通信するにはマスターが最初にスレーブアドレスを送信し、続いて複数のデータを送信ないし受信する。LCD のアドレスは `0x3e` である。

I2C のデバイス名は `/dev/i2c-1` である。これをオープンすることで I2C 通信が可能となる。(なお, Rasp-

berry Pi には 2 本の I2C があるが、1 本は GPU に使われているため/dev/i2c-0 はデバイスとしては存在しない.)

プログラムの骨格は次のようになる (H.1 も参照).

```
i2c=open("/dev/i2c-1", O_RDWR);
val=ioctl(i2c, I2C_SLAVE, 0x3e); // I2C 独自の設定
initLCD(i2c); // lcd.c の中に定義されている (戻り値なし)
val=lcd_datawrite(i2c, "hello"); // この関数も lcd.c 中で定義
val=close(i2c);
```

また、LCD にコマンドを送ることで表示の制御が可能である。Apple Pi の LCD モジュールは AQM0802 であり、マニュアルは~/pj/pdf/AQM0802.pdf である。この中の Display Instruction Table がコマンド一覧である。これを見ると、例えば画面消去のコマンド "Clear Display" が 0x01 であること、このコマンド完了のために (OSC が 540KHz のため) 1ms 程度待たなければならないことが分かる。これをプログラムで書いたものが lcd.c の中にある関数 clear である。この関数定義を読んで動きをよく理解すること。(なお、AQM0802 内部の制御チップは ST7032i であり、ST7032i のマニュアル~/pj/pdf/ST7032.pdf には全機能の詳細が説明されている.)

小問題 3:

配布されている~/pj/devices/lcd.c には main の中身がないので、以下のような動作になるように lcd.c を完成させなさい。

- 上の説明を参考にエラー処理を追加する。(lcd_cmdwrite と lcd_datawrite はエラーの時は-1 を返す.)
- "hello" と 1 秒表示して画面を消去し、次に "world" と 1 秒表示し画面を消去する。以上を繰り返すプログラムを作成しなさい。(1 秒待つのはタイマーでなく sleep を使ってよい.)

(本当に簡単な余力問題) 下の行にも文字を表示せよ。表示する行を変えるには、lcd.c に定義済みの location という関数を使う。第 2 引数が 0 の時は上の行、1 の時は下の行に表示位置が変わる。

(余力問題 1) Display Instruction Table を読み、ディスプレイを 0.5 秒に 1 回点灯させるようにせよ。(Clear Display とは異なり文字は消去せずに、Display を ON/OFF する.)

(余力問題 2) Raspberry Pi の I2C には、bme280 という温湿度/気圧センサーも接続されている。bme280.c は温度と気圧を測定するプログラムである。BME280 のマニュアル(~/pj/pdf/BST-BME280.pdf)を読んで湿度測定を bme280.c に追加せよ。なお、マニュアルに記載されている湿度補正の関数は、既に bme280.c に含まれているので自分で再入力する必要はない。(このため、実際に追加するプログラムは数行である.)

4.5 デバイスの操作 - GPIO デバイス

■ mmap とは何かを説明する。

■ 16 進表現とビット演算子 (&, |, ~, <<) の説明。ビットの立て方、降ろし方。(上級者向け。使わなくてもプログラムは書ける.)

上で簡単に LCD を動かすことができたのは、OS の中でドライバというソフトが動いているからである。実は/dev/i2c-1 はファイルの名前ではなくドライバの名前であり、これを open して write するとドライバに対する指示となる。ドライバは便利だが、ハードウェアを隠蔽してしまうのでハードウェアを制御する方法が理解しにくくなる。ここではドライバを使わずに直接ハードを制御するプログラムを書いてみる。

メモリマップド IO とは、CPU に接続されたハードウェアを制御する方法の一つである。メモリマップド IO では、ハードの制御レジスタがメモリ内のある番地に置かれており、そこに読み書きすることで制御をする。Raspberry Pi には GPIO という接続方法で LED やタクトスイッチが接続されており、以下では GPIO をメモリマップド IO で制御する。

メモリマップド IO の制御とは物理メモリを直接読み書きすることである。これには mmap というシステムコールと /dev/mem というファイルを使う。/dev/mem は実はファイルではなく、物理メモリ全体について名前である。これを mmap することで物理メモリにアクセスできる。次のようなプログラムとなる。

```
void *gpio_baseaddr;
fd = open("/dev/mem", O_RDWR);
gpio_baseaddr = mmap(NULL, 4096, PROT_READ | PROT_WRITE, MAP_SHARED,
                      fd, GPIO_PHY_BASEADDR);
```

GPIO_PHY_BASEADDR の値は 0x3F200000 である。これが GPIO の制御レジスタが存在する物理アドレスである。fd (つまり /dev/mem) の何バイト目から mmap するか指定している。上の mmap により、gpio_baseaddr から始まる 4096 バイトに GPIO の制御レジスタが置かれたように見える。あとはその制御レジスタを読み書きすれば GPIO を制御できる。

タクトスイッチは GPIO の 22~27 番に接続されている。GPIO の信号線がオンかオフかは、gpio_baseaddr + GPIO_GPLEV0 番地に存在するレジスタで見ることができる。つまり、次のようなプログラムとなる。

```
unsigned int x;
x = *((unsigned int *) (gpio_baseaddr + GPIO_GPLEV0));
printf("%x\n", x);
```

これにより、x に GPIO の 0 番~31 番までの値が入り、16 進で表示される。x の第 0 ビットが GPIO0 に、第 31 ビットは GPIO31 に対応する。

なお、上の定数などをまとめたファイルが ~/pj/devices/gpio.c である。次の課題ではこのファイル（の中身）を使う。また、この中には関数 memread と memwrite も入っている。ポインタとキャスト（型変換）の扱いに慣れている人は上の書き方でよいが、そうでない人は memread と memwrite を使ってメモリを読み/書きしてもよい。やっていることは同じであるが、memread/memwrite の関数定義の方が読みやすいかもしれない。

小問題 4:

~/pj/devices/gpio.c をベースにして、GPIO の値を表示するプログラム gpio.c を作りなさい。タクトスイッチは GPIO に接続されているため、GPIO の値を見るとスイッチが押されているか離されているかが分かる。GPIO の値を見るには gpio.c の中にある memread という関数を使うとよい。（上に書いてある方法で、ポインタで直接見てもよい。）

なお、タクトスイッチが押された時の GPIO の値を見るには、スイッチを押したまま a.out を実行する。

mmap を用いたファイルの入出力

mmap は実はメモリマップド IO よりもファイルの入出力のために使うことが多い。ファイルをメモリに map することで、ファイルの中身を配列として読むことができる。

```
char *baseaddr;
fd = open(ファイル名, O_RDONLY);
baseaddr = mmap(NULL, ファイルの長さ, PROT_READ, MAP_SHARED, fd, 0);
printf("%c\n", baseaddr[0]);
close(fd);
```

これにより、ファイルの先頭の文字が表示される。上は読む時の例なので書きたい時は自分で調べること（G.5 も参照）。

GPIO からの出力

デフォルトでは GPIO は外部の電圧を入力する設定になっている。Raspberry Pi から外部に電圧を出力するには、入出力の設定を変える必要がある。この設定は GPFSEL0 で行う。GPIO5 番には LED が接続されているので、これを点灯してみよう。次のようなプログラムとなる。(類似のプログラムが gpio.c の中に入っている。)

```
unsigned int gpfsel0 = *((unsigned int *) (gpio_baseaddr + GPIO_GPFSEL0));
gpfsel0 = gpfsel0 | (1<<15); // 15=GPIO5*3
*((unsigned int *) (gpio_baseaddr + GPIO_GPFSEL0)) = gpfsel0; // 出力設定
*((unsigned int *) (gpio_baseaddr + GPIO_GPSET0)) = (1 << 5); // 点ける
sleep(1);
*((unsigned int *) (gpio_baseaddr + GPIO_GPCLR0)) = (1 << 5); // 消す
```

詳細は付録 H.3 参照。

第 2 回提出課題

茶ボタン (GPIO22 番) を押すと LCD (の上の行) に”hello”と表示し、ボタンを離してから 1 秒経ったら消すようなプログラム keydisp.c を作成しなさい。以下を提出すること。

- keydisp.c (特に注意すべき行については、行の右側にコメントを書く)
- 全体の動作の説明を書いたドキュメント (docx, txt, pdf 等)

最低でも以下を書くこと

- ボタンが押されていることと離されていることをどう検出しているか
- (その検出結果を使って) ボタンが押されたことと離されたことをどう判定しているか
- 「離してから 1 秒経ったら消す」をどのように実現しているか
- LCD への表示と消去はどのように実現しているか
- 余力問題を解いた人はその部分についても説明する
- 開発環境の工夫があれば上記ドキュメントに追加記述する

以下の注意点を守ること。

時間経過はタイマーで計測すること。なお離したことを検出してからタイマーをスタートすること。押していることを最後に検出したときから 1 秒経過したことを測るのではなく、離したことを検出してから 1 秒経過したことを測定しなければならない。(上の「動作の説明」の書き方にも関連するので、よく理解すること。)

もう一つの注意点はシグナルハンドラ内の処理内容である。LCD はコマンドを送ったら何 ms 待つなどの時間制御が伴う。タイマーでシグナルが発生すると、コマンド送信途中などの微妙なタイミングでシグナルハンドラが実行される可能性がある。このためシグナルハンドラの中では LCD 制御は行ってはならない。シグナルハンドラ内ではグローバル変数への代入を行い、メインのループでその変数をチェックして LCD 制御などをする。これは機器制御をする際の基本的なテクニックである。

ヒント 1: GPIO0 番から GPIO31 番までが 1 つの int の中に入っている。このうち GPIO14 番と 15 番はリアルに接続されており、タイミングによってはそのビットも変化するかもしれない。本来はビット演算を使って GPIO22 番だけを見るべきであるが、GPIO22 番以外のビットは変化しないと思って作っても実際はほぼ問題ない。

ヒント 2: 文字を消すには clear 関数を使ってもよいが、余力問題 2 をやるなら連続したスペース" "を表示して消した方がよい。

ヒント 3: lcd.c の中では I2C のファイル記述子はローカル変数だが、グローバル変数としても構わない。ロー

カル変数のままでも実装可能なのでどちらでもよい。

余力問題 1: 1 秒経つ前にまた押したら文字は消さず、最後に離れた時から 1 秒経ったら消すようにしなさい。

余力問題 2: 茶ボタンの動作に加え、赤ボタン (GPIO23 番) を押したら、下の行に "world" と表示し、赤ボタンを離してから 1 秒経ったら消すようにしなさい。この時、赤と茶は独立して動作すること。つまり、一方のボタンの 1 秒カウントを行っている途中に、もう一方のボタンを操作しても影響を受けないようにする。

余力問題 3 (かなり余力のある人用の問題): 上の余力問題 1 と 2 を組み合わせる。つまり、余力問題 1 が茶と赤で完全に独立して動くようにする。(かなり面倒です。)

問題とは関係ないが、"hello" や "world" ではなく温度と気圧を表示すれば少しは実用的かもしれない。

【開発環境の工夫 (若干の加点あり)】

この課題を作るにあたっては「作り方」をよく考えること。いきなり全体を作り始めるとかなり手間が掛かる。本当にターゲットでしかできないこと (LCD の表示と GPIO の入力) だけはターゲットで作らざるを得ないが、時間計測を含む全体の流れはできるだけホストで作る。LCD の表示は表示用の関数を作り、中では printf で模擬すればよい。GPIO の入力は入力用の関数を作って、例えば 1000 回呼び出されるまで 0 を返し、2000 回呼び出されるまでは 1 を返すなどと模擬 (シミュレート) すればよい。ターゲットで動かす本物の表示用関数や入力用関数はターゲットで動作確認をして、最後に組み合わせれば全体は一発で動作するはずである。

クロス開発においてターゲットの環境をホスト側で模擬して開発することは、とてもよく使うテクニックである。これによって劇的に開発時間が短縮される。急がば回れである。分割コンパイルを使うと更に楽になる。本体は keydisp.c とし、LCD やボタン用の関数が入ったプログラムはホスト用に host-io.c、ターゲット用に target-io.c を作る。cc をする時にファイル名の指定を変えるだけでよい。もちろん、Makefile を作ればもっと楽になる。

メモ: 自宅で作成するときは Raspberry Pi とノート PC を LAN ケーブルで直結しても動作する。(当然だがインターネットには接続できない。)

デバイスを操作する別の方法:

実はプログラムを作らなくとも `/sys/class/gpio/` を使って GPIO を簡単に操作できる。(ただしあくまで実験用であって、例えば複数人で同じ GPIO を使うと衝突して予期せぬ動作となる。)

```
$ echo 5 > /sys/class/gpio/export
```

と実行すると `/sys/class/gpio/gpio5` の下に次のようなファイルが作られる。(以下の説明で `r` は `read`, `w` は `write`..)

- `direction`: 「入出力方向の状態」の読み出し (`r`), 設定 (`w`). "in" で入力, "out" で出力.
- `value`: 「現在の状態」の読み出し (`r`), 書き込み (`w`). "0" または "1".
- `edge`: 「割り込み状態」の読み出し (`r`), 設定 (`w`). "none", "rising", "falling", "both".
- `active_low`: 電圧が高い時に `value` が 1 になるか 0 になるかの設定 (`w`)

次のようにして実験してみる.

```
# cd /sys/class/gpio
# ls                ← gpio5 というディレクトリがないことを確認
# echo 5 > export
# ls                ← gpio5 というディレクトリが作られたことを確認
# echo out > gpio5/direction
# echo 0 > gpio5/value
# echo 1 > gpio5/value
# echo 5 > unexport
# ls                ← gpio5 というディレクトリが削除されたことを確認
```

1 秒毎に LED が点滅するような shell スクリプト `led` を作成しなさい. なお, `bash` の無限ループは次のように書く.

```
while true
do
    処理内容
done
```

もう一つの LED は GPIO6 に接続されているので, 1 秒毎に LED が交互に点滅するようなプログラムを作っても面白い.

`/sys` についてのメモ: デバイスの制御はドライバを介して行うのが基本であるが, パラメータの設定など, いちいちドライバを使わずにやりたいことがある. Linux の `sysfs` という機能は, デバイスの現在の状態をファイルとして実体化しており, ファイルのようにデバイスを簡単に操作できる. これが `/sys` というディレクトリの下にある各ファイルである. なおこれは `/proc` と同じく, 実際にファイルとして存在しているわけではなく, 仮想的なファイルである.

5 第 5 回

今回の内容:

- 第 2 回提出課題の回答を簡単に説明

- TCP/IP のレクチャ
- ソケットによる通信プログラム
- 簡単な Web サーバの作成（前編）

5.1 TCP/IP とソケット

まず TCP/IP の説明を行う。「情報ネットワーク」の授業でも TCP/IP について説明があるが、ここではプログラミングに近い部分を中心に説明する。

以下のキーワードを理解すること。

- MAC, IP, TCP/UDP, アプリ
- MAC アドレス, IP アドレス, ARP, DNS

ネットワークの仕組みを調べるツールには、例えば次のようなものがある。

- `ip a` (address の略): 自分の IP アドレスを調べる
- `ip n` (neighbor の略): ARP 表の表示. 削除もできる.
- `ping`: ICMP パケットの送信. 到達性確認の基本ツール.
- `host`: ホスト名から IP アドレスを引く. IP アドレスからホストも引ける.
- `traceroute/tcptraceroute`: 経路情報の表示 (学外へは `tcptraceroute` のみ動作).
- `whois`: ただし学内ではファイアウォールで動作しないため `http://whois.jp.rs.jp/` を使う.

簡単な実験:

実験 1:

1. 現在の ARP 表を表示する.
2. 近くの人の IP アドレスを聞き, そこに `ping` を打つ.
3. ARP 表を再表示し, エントリが追加されたか確認する.
4. 相手のマシンの ARP 表もエントリが追加されているか確認する.
5. 存在しない IP アドレス (例えば 172.28.34.99) に `ping` を打ってみる.

実験 2:

- `www.shibaura-it.ac.jp` の IP アドレスを調べてみる.
- それ以外に適当なホスト名の IP アドレスを幾つか調べてみる.
- `http://whois.jp.rs.jp/` にアクセスして, 適当なドメイン (例えば "shibaura-it.ac.jp") の「ドメイン名情報」を検索してみる.
- `www.shibaura-it.ac.jp` への経路を調べてみる.

5.2 ソケットによるプロセス間通信

サーバ, クライアント, TCP, ソケット, ポートの説明をする. TCP により信頼性のある通信が実現される. 通信は双方向, データ長は無制限だが, 開始と終了がある. TCP の仕組みにより, 必然的にリアルタイム性は保証されない. またフロー制御がある. 例えば, 受信側がデータを受信 (read) しないとやがて送り側がストップする. (なお, TCP 以外のプロトコルに UDP があるが, この授業では取り上げない.)

プロセス間通信の一つであるシグナルについては 1.5 節で取り上げた. 今回は, 別のプロセス間通信方法として, ソケットを取り上げる. これまでと同様に, ソケット通信を待ち受ける側をサーバ, 通信要求を発行する側をクライアントと呼ぶ. サーバ側のやるべきことをプログラムの骨格だけ書くと次のようになる (例によってエラー

チェックを省略している).

```
#include <arpa/inet.h>          // これ以外の include は自分で調べる
struct sockaddr_in serv_addr;

// ソケットを作る
sockfd = socket(AF_INET, SOCK_STREAM, 0);

// アドレスを作る
memset(&serv_addr, 0, sizeof(struct sockaddr_in));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(ポート番号);

// ソケットにアドレスを割り当てる
bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr_in));

// コネクション要求を待ち始めるよう指示
listen(sockfd, 5);

// 要求があったらそれを受け付ける (なければ待つ)
new_sockfd = accept(sockfd, NULL, NULL);
// クライアントからデータを受け取る
read(new_sockfd, buff, 128);
// 1 秒待ってソケットを終了する
sleep(1);
close(new_sockfd);
close(sockfd);
```

一方, クライアント側のプログラムは, 次のようになる.

```
// ソケットを作る
sockfd = socket(AF_INET, SOCK_STREAM, 0);
// サーバのアドレスを作る
memset(&serv_addr, 0, sizeof(struct sockaddr_in));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr("172.28.34.~");
serv_addr.sin_port = htons(ポート番号);
// コネクションを張るための要求をサーバに送る
connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr_in));
// 128 バイトのデータをソケットで送る
write(sockfd, buff, 128);
// ソケットを終了する
close(sockfd);
```

各システムコールは次のような意味である.

- **socket**: ソケット (ファイル記述子) の生成

- `bind`: ソケットにアドレスをつける
- `listen`: コネクションの要求を待つ
- `connect`: コネクションを張る要求を送る
- `accept`: コネクションの要求を受け入れる
- `write, read`: データの送受信
- `close`: ソケットを閉じる

各ライブラリ関数の詳細な使い方は自分で調べる。close の手順には注意が必要である。クライアントが close してからサーバが close しなければならない。ここでは手抜きの方法として、サーバが 1 秒待ってから close している。正しく close する方法については、G.6.1 を必ず参照すること (重要)。

小問題 1:

まず、クライアント (ターゲット) から任意の数字をサーバに送り、サーバでそれを表示するプログラムを作成せよ。

必ずすべてのシステムコールに対して、エラーチェックを入れること。エラーの時にどのような値が返されるかは、man で調べる。

クライアント用プログラムは `cli.c`、サーバ用プログラムは `serv.c` とする。ポートは、50000 番を使うものとする。(ポート番号 49152~65535 は自由に使える。) クライアント・サーバ間で通信する数字は、最初は `char` で良い。(char も数字だったことに注意。ただし、範囲は-128~127。) つまり、上のプログラムで言えば、`buff[0]` だけを使ってデータを通信する。

ターゲットに実行可能ファイルを毎回転送するのは大変なので、ホスト上でサーバとクライアントの両方のプログラムを作てよい。(なお LAN に接続しないでプログラムを作るときは、サーバの IP アドレスとして“127.0.0.1”を指定する。これは自分を意味する特殊な IP アドレスである。)

余力がある人は、以下について考えてみる。

- クライアント (ターゲット) から任意の数字をサーバに送り、サーバ (ホスト) 上でそれに 1 を加えてクライアントに戻すようなプログラムを作成せよ。
- 1 回の `read` と `write` で 1,000,000 バイトを送受信してみて動作を調べる。どのようにすれば 1,000,000 バイト全部を通信できるだろうか。
- サーバが一回で終了せず、何回もクライアント接続が可能にする。
- `htonl` や `ntohl` という関数が何のためにあるかを調べる。

send と recv:

上の例では、ソケットへの送受信を一般的な `write` と `read` で実行したが、通信専用のカーネルコールとして `send` と `recv` がある。`send` と `recv` では引数が追加されており、これを使って詳細な動作指定ができる。例えば、待ちに入らず (つまりブロックせず) に直ちにリターンせよというような指定ができる。

5.3 簡単な Web サーバーの作成

HTTP と HTML について説明する。HTTP は、HTML を転送するためのプロトコルである。クライアントが HTTP リクエストを送信すると、サーバが HTTP レスポンスを送信する。HTTP リクエストは、リクエスト行、ヘッダフィールド、ボディから構成される。HTTP レスポンスは、状態行、ヘッダフィールド、ボディから構成される。

HTML は、Web ページを記述するための言語である。HTML の詳細は本演習の対象外なので、ごく簡単に説明する。

小問題 2:

次のようなサーバーを作れ。ファイル名は webdump.c とする。サーバーは 80 番ポートで待つ。ソケットが接続したら、ソケットからデータを入力しては、その内容を画面に書くことを繰り返す。

ただし、これまで入出力は read と write を使ってきたが、上で説明したように HTTP では人間が読める文字列で通信が行われる。このため、read と write でなく、fgets や fprintf で入出力をした方が、プログラムがずっと読みやすくなる。しかし、fgets や fprintf はファイル記述子に対して直接入出力をすることはできず、ストリームに対してしかできない。そのため fdopen を使って、ソケット（ファイル記述子）をストリームに変換する。つまり、次のようになる。

```
FILE *istream;
istream = fdopen(ソケット, "r+");
setvbuf(istream, NULL, _IONBF, 0);
```

fdopen の引数の意味は自分で調べる。setvbuf は後でこのストリームに対して fprintf をする時に、ただちに送信するための指定である。これを指定しないと、fclose するまで送信しない可能性がある。

fgets は今まで使ったことがないので、念のためプログラムのメイン部分だけ示す。buff は適切に宣言されているとする。

```
while (1) {
    if (fgets(buff, 1024, istream) == 0) break;
    printf("%s", buff);
}
```

なお終了時には、fdopen したストリームをクローズすること。（これは重要。クローズしないと送信しないことがある。）なお元のソケットをクローズすると正しく動作しない。

このサーバーは **80 番を使うので特権ユーザ** で実行する（1023 番以下はプロトコルが決まっているポートなので、一般ユーザは使えない）。そして（同じマシンの）ブラウザからアクセスしてみる。サーバーからの出力を見れば、ブラウザから送信されているデータが分かる。もし自分のスマホが大学の無線 LAN に接続している場合は、同じ URL にアクセスしてみて、出力を比較してみてもよい。

なお、サーバーからレスポンスが返されないため、ブラウザはハングした状態になり、サーバーもそのまま停止する（上の fgets で待ったままになる）。サーバーを ^C で止めると、ブラウザもエラーで止まるはずである。もしくはブラウザを閉じてしまえば、サーバーも停止するはずである。

課題:

上記のサーバーをベースとして、次のような動作をするように変更する。ファイル名は webget.c とする。

大きな変更点としては 2 つである。第 1 は HTTP レスポンスを返すことである。まず HTTP GET リクエストを最後まで読み込む。（ヒント: HTTP GET はヘッダしかないリクエストだから、ヘッダの終わりまで読めばそれが最後まで読んだということである。そして、ヘッダの終わりはある特定の行によって表現されていたことを思い出そう。）最後まで読み込んだら while ループを break で抜けて、レスポンスを返す。レスポンスを返すにはストリームに対して fprintf を実行すればよい。レスポンスとしては最小限のヘッダだけとし、ボディは短くてよい（hello\r\n など）。ここで注意すべきは、一行の終わりは \n ではなく \r\n であることである。fprintf に必ず \r\n を付けること。

第 2 の変更点は、サーバーを終了させないことである。世の中のすべての Web サーバーは、一回通信が終わったからといって終了はしない。次の通信を待ち受けることを繰り返す。上記の流れの中で、accept から fclose までの部分を無限に繰り返すようにする。（注意: listen は一回実行すれば十分である。）

（余力のある人）リクエストの 1 行目に GET の対象となるパス名が入っているので、この部分を抜き出す。こ

れには付録 G.7 の `get_filename` を使うとよい。(`get_filename` の引数は `'/'` で始まる文字列を想定しているので、その文字を探してそこへのポインタを渡せばよい。) 抜き出した文字列をファイル名とし、そのファイルの内容をレスポンスとする。ファイル内にはヘッダとボディをすべて書いておき、それをそのままクライアントに返すのが簡単である。(本当の HTML ファイルにはボディだけが書かれていて、ヘッダはサーバーが生成するが、今回はその部分をサボることにする。) ただし、ファイルにおいて一行の終わりは `\n` である。一方、ネットワークに対しては `\r\n` を送信しなければならない。これには例えば次のような方法が考えられる。(`fgets` した文字列が `buff` に入っているとするとする。)

```
char *p = strchr(buff, '\n');
if (p!=0) *p = '\0';
fprintf(stream, "%s\r\n", buff);
```

ブラウザによっては `favicon.ico` をリクエストして来るかもしれない。これが何かは自分で調べるとして、エラーにしないためには空の `favicon.ico` ファイルをする。次のようにして `favicon.ico` を作成する。

```
$ echo -n > favicon.ico
```

6 第6回

今回の内容:

- ネットワークのツール
- 簡単な Web サーバーの作成 (後編)
- 最終課題

6.1 ネットワークプログラミングのためのツール

ネットワークプログラミングを支援してくれるツール群を幾つか紹介する。

- `ss`: ソケット一覧表示

```
ss -tn    TCP ソケットの一覧
ss -lpn   listen 中ソケットの一覧
ss -lpn "sport = :80"  80 番ポートを listen 中のソケットの一覧
```
- `lsof`: open 中のファイルの一覧

```
lsof -p PID    PID プロセスがオープンしてるファイル
lsof ファイル名  指定ファイルをオープンしているプロセス
lsof -i :50000   50000 ポートのソケットをオープンしているプロセス
```
- `tcpdump`: パケットダンプ (特権ユーザで実行)

```
tcpdump -i lo -X   ホスト内の通信のダンプ
tcpdump -X port 80 and host 172.28.34.60   ポートとアドレスを指定したダンプ
```
- `telnet`: ソケットを張り、手で直接入出力できる
- `wget`: HTTP でページをダウンロードする

小問題 1:

前回のプログラムの通信状況をモニターしてみる。

1. `webget.c` を走らせて `ss` コマンドで本当に 80 番を listen してるかを見てみる。
2. 前回のように自分のホスト内で `webget.c` とブラウザを動かし、これを `tcpdump` を使ってダンプする。

3. webget.c を動かし、右隣の人にブラウザでアクセスして貰う。これを `tcpdump` を使ってダンプする。
4. ポート番号を指定したダンプなどを試してみる。

6.2 Form/HTTP POST の仕組み

HTML Form の仕組みと、それによって発行される HTTP POST プロトコルについて説明する。

実験:

名前を入力する FORM タグを含む Web ページ `form.html` を HTML で記述せよ。form を submit した結果のページは `post.html` とする。また `method='post'` とする。前回のプログラム (`webget.c`) を使って通信の中身を見る実験をする。HTTP リクエストのリクエスト行 (つまり 1 行目) がどのようなかを確認する。手順の詳細は、前回の余力問題 (`get_filename` を使うところ) までできたかどうかで異なる。

[`get_filename` ができていない人] 自分のホームディレクトリに `form.html` を記述する。この時、

```
action='http://127.0.0.1/post.html'
```

とする (これは一般的な action の書き方ではないので注意)。form.html が完成したらブラウザを立ち上げ、URL として

```
file:///home/自分のアカウント名/form.html
```

と入力する。これで `form.html` が表示されたはずである。webget.c を動かして、ブラウザで form に何らかの文字列を入力して、送信ボタンを押す。

[`get_filename` ができている人] `form.html` にヘッダも含めて書く。webget.c を動かし、ブラウザに `form.html` の URL を入力する。これで `form.html` がブラウザに表示されたはずである。form に何らかの文字列を入力して送信ボタンを押す。

前回の通りに作ってあるとヘッダの部分は表示されるが、空行から下の部分は表示されない。これは次の小問題で解決する。

(余力のある人) `method='get'` として同じことをしてみる。form に入れたデータが post とは違う形で送信されたのが分かる。(GET でフォーム入力の値を送ることは、あまり推奨されない。)

小問題 2 (第 6 回の提出課題) :

まず、`form.html` で `method='post'` にする。次に前回のプログラム `webget.c` をベースにして、FORM の submit に対応するようにせよ。ファイル名は `websrv.c` とする。`webget.c` と `websrv.c` の違いは次の点である。

[`get_filename` ができていない人]

1. モード用の変数を用意する (`int` でも `char` でも適当に)
2. GET メソッドが来たら、モードを GET にする (`strncmp` を使う)
3. POST メソッドが来たら、モードを POST にする
4. Content-Length: のヘッダが来たら、その残りの文字列を整数に変換する (下の注意 2 を参照)。
5. ヘッダが終了したら、GET モードの時はヘッダと `form.html` の内容を `fprintf` を使ってクライアントに返す。POST モードの時は `fgets` を使って Content-Length の値 +1 の長さを読み込んで画面に表示し、`fprintf` を使って適切なヘッダと "OK" 等の適切な文字をクライアントに返す。(なぜ +1 するか分からない人は `man fgets`)

注意 1: `fprintf` で " " を含む文字列を出力したい時は、`"\"` と書く。

注意 2: 文字列の数字を `int` の値に簡単に変換する方法は以下の 2 つ:

- `atoi` を使う. `atoi` の引用にその文字列の先頭のアドレスを渡す必要がある. なお, `atoi` は数字の後に `"\r\n"` が付いていても無視してくれる.)
- `scanf` で文字列を読み飛ばすテクニックを使う. 例えば `scanf("abcde: %d",&x);` と実行して, `abcde: 100` と入力すると, `abcde:` の部分は飛ばして `x` に数値の 100 が入る. 後はこのテクニックを `sscanf` で使えばよい. `sscanf` と `scanf` の違いは自分で調べる.

[`get_filename` ができている人]

1. モード用の変数を用意する (`int` でも `char` でも適当に)
2. GET メソッドが来たら, モードを GET にする (`strncmp` を使う)
3. POST メソッドが来たら, モードを POST にする
4. どちらの場合も `get_filename` を使ってリクエスト行からファイル名を抜き出す.
5. `Content-Length:` のヘッダが来たら, その残りの文字列を整数に変換する (上の注意 2 を参照).
6. ヘッダが終了したら, GET モードの時は元々の動作をする (つまりファイルの内容をクライアントに返す). POST モードの時は `fgets` を使って `Content-Length` の値 +1 の長さを読み込んで画面に表示し, あとは元々の動作をする. (なぜ +1 するか分からない人は `man fgets`)

(余力問題)

- `input type=` に `radio`, `checkbox`, `select` などを指定した時に送信されるデータを, `websrv.c` を使って調べよ.
- POST メソッドによって送られたユーザ入力データを含むような `html` を返すように修正せよ. (例えば `address` に `tokyo` と入れたら「Your address is tokyo」などと返す.)

6.3 最終課題

通信プログラム (2 種類), 組み込みプログラミング (1 種類), 並行プログラミング (1 種類) があり, どれか一つを選択して作成する. 自分のオリジナルな機能は勝手に入れてもよい (加点対象).

6.3.1 マルチスレッド Web サーバー (低難度)

第 5 回, 6 回で作成した Web サーバーを高度にする. クライアントが複数あった時, あるクライアントに対するソケットにおいて入力待ちが発生すると, 他のクライアントの処理が停止してしまう. これに対処する方法は幾つかある. ここでは, 一つのクライアントに対して一つのスレッドを作る方法で対処する. あるスレッドが入力待ちになっても, 他のスレッドは停止しないで実行継続できる. (作成したスレッドはいつ終了するか分からないので `pthread_join` は難しい. このため `pthread_detach` で切り離すこと.) そのような Web サーバーを `websrv.c` をベースにして作成する.

また, 単に指定したファイルの内容を返すのではなく, Raspberry Pi の温度計で温度 (湿度や気圧でも構わない) を測定し, それを含む HTML を返すものとする.

なお, これをそのまま実装しても, 本当に複数スレッドが並行動作しているかすぐには分からない. レスポンスを返す前に 10 秒のスリープを入れるなどして, 無理に時間の掛かる状況を作り出し, クライアントを複数同時に動作させて動作確認すること.

(以下は加点対象の追加機能) この時, 複数のスレッドが同時に温度計にアクセスしないように `mutex` で排他制御まですると, さらに良い.

6.3.2 IO 多重化 (高難度)

上記のマルチスレッド型 Web サーバーの方法では沢山のスレッドを作る必要がある. 待ちに入るのが問題なのだから, 待ちを回避できさえすればよい. これを IO 多重化と呼ばれる仕組みを使って実現する. 具体的には,

select, poll, epoll のいずれかを用い、マルチスレッドやマルチプロセスは使わないサーバープログラムを作成せよ。こちらは必然的にシングルスレッドになるので、通信 1 番のような排他制御は必要ない。(なお、上記のシステムコールはファイル記述子を扱う。fdopen でストリーム化したソケットとファイル記述子のソケットが混在することになる。混在しても正しく動作するはずだが、マニュアルには明記されていないようである。何らかの不具合が出る可能性もある。)

サーバーとして何を作るかは自由に決めてよい。ただし、ソケットを張りっぱなしにして何回か処理をするようにしないと、本当に IO 多重化が動作しているか示せない。例えばある数値を送ったらサーバーが演算(階乗とか)をしてそれを返すことを繰り返す、クライアントが負の値を送ったらクローズするなどが考えられる。もう少し面白い例はチャットサーバーである。チャットが続く限りはソケットを張りっぱなしにする。Web サーバーでも、HTTP の Keep-Alive を実装すればソケットが張りっぱなしとなる。これができるば、かなり本格的な Web サーバーを作ったことになる。

6.3.3 ゲーム(作る機能により低~中難度)

第 4 回で修得したハード制御の知識を組み合わせるゲームを作成する。自分がゲームだと思えば何でもよいが、キー入力と LCD を使った組み込みプログラムであること。具体的には以下の機能を最低 3 つは使うこと。

- タクトスイッチ(ボタン)を 2 個以上使う
- タクトスイッチの同時押しを判定する
- 温度計・湿度計・気圧・LED などを使う
- 任意の位置への文字表示(ヒント: 関数 location をよく読む)
- 左右スクロール(AQM0802 の用語では Shift)
- 縦長フォント
- コントラストの変更
- CGRAM(H.1 参照)
- タイマー
- タイマでないシグナル
- スレッド(なおスレッドとシグナルは同時に使わない方がよい)

6.3.4 銀行間の並行プログラミング(中難度~高難度)

第 3 回で作成したトランザクションシステムを高度化する。以降ではトランザクションは Tx と書く。どの問題も trans.csv を生成するプログラム~/pj/txn/txgen.c を修正する必要がある。まず txgen.c を読んで理解しておく。なお使い方は(txgen にコンパイルしたとして)、以下のように実行し、この結果をリダイレクションで trans.csv に入れば良い。

```
$ ./txgen トランザクション数 口座数 最大金額
```

2 つの銀行を作り、銀行内の振替と銀行間の振込を処理するプログラムを作成せよ。各銀行は異なるマシン上に作ることを想定しているが、Web サーバ開発のときのように開発は 1 台のマシン上で行って構わない。

txgen.c が次のような出力をするように修正する。

```
t,1234,5678,1000
o,1234,5678,1000
```

'o' は別(other)の銀行の意味で、自分の 1234 番から other の銀行の 5678 番へ 1000 円を振込むという意味である。ここでは銀行は 2 つ(自分と相手)しかないので、other とはつまり相手の銀行のことである。't' と 'o' の出現は乱数で適当に決める。trans.csv は銀行毎に(つまり 2 つ)作る。

両方の銀行(マシン)で同じように Tx の処理をする。各銀行ではスレッドを 1 つ作成する。main(こちらを送金スレッドと呼ぶ)では自分の銀行用の trans.csv を読み込み処理をする。振替('t' の Tx)は通常通り処理

する。振込（'o' の Tx）は、まず自分の銀行内の口座を減額する。相手の銀行に振込金額の情報を送り、返事が返ってくるのを待つ。完了の返事が返って来たらその Tx の処理が完了する。

別スレッド（受金スレッドと呼ぶ）は相手からの通信を待つ。振込情報が送られてきたら、対象の口座に振込を実行（つまり加算）し、実行完了を示す返事をする。一つの銀行内の2つのスレッドは同じ口座を処理する可能性があるため、mutex を1つ作り、衝突しないようにする。

まとめると2つの各銀行内に2つのスレッド（main スレッド=送金スレッド、受金スレッド）があり、一方の送金スレッドともう一方の受金スレッドが互いにソケットで接続される。送金スレッドは振替の処理は自分でを行い、振込は相手と通信して処理する。

ヒント 1: 数値を通信でやり取りするが、`fscanf(istream, "%d", ~);` などのように直接ストリームから読まない方がよい。Web サーバの時のように、`fgets` で一行丸ごと読むことを強くお勧めする。その後で `sscanf` を使って `char` の配列から数値を読み取る。（`fscanf` は改行を読まないで予期せぬ動作となる。）

ヒント 2: サーバとクライアントはほとんど同じ動作をする。これを何も考えずに作ると同じコードを2回書くことになり、二度手間の上にデバッグも困難になる。お勧めの作り方の一つは、一つのプログラムとして作ってオプションで動作を変更する方法である。

```
$ ./a.out s
$ ./a.out c
```

このようにして動作を切替える（もちろん前者がサーバ動作、後者がクライアント動作）。オプションの値を見る方法は G.2 を参照のこと。あるいは分割コンパイルを使って、共通部分を別ファイルに抜き出す方法でもきれいに書ける。

この問題には次のような拡張が考えられる（加点対象）。

- 口座が残高不足だったらその Tx は実行しないで後で再実行するようにせよ。そのためには全体を繰返し実行するようにして、一回の繰返しで一つの Tx も実行しなかったら終了とする。相手側で Tx を再実行したために自分の Tx が実行できるようになる可能性もあるので、相手と揃えて繰返しを実行して行き、双方のカウントの和で終了判定する必要がある。
- 上の通りに作ると相手銀行の処理と自分の処理を別々に実行するので、実は正しくない結果になる可能性がある。本来は相手の mutex と自分の mutex の両方を取ってから振込の処理をすべきである。ただし、そのようにするとデッドロックする可能性がある。回避方法の一つは、タイムアウト付きの lock（`pthread_mutex_timedlock`）を使い、しばらくロックが取れなかったら一旦あきらめて lock を解放し、少し待ってまた2つの mutex の lock に挑戦するという方法である。これを実装せよ。
- 上記は振込む方向と通信の方向が同じであった。振込まれる側（受金側）からも振込情報を送れるようにせよ。簡単にはマイナスの金額を送れると考えればよいが、送金側が残高不足のことがある。
- 各銀行内をマルチスレッド化する。なお1本のソケットに複数のスレッドが read/write すると正しく動作しない。送金スレッドと受金スレッドは同じ個数だけ作り、相互にソケットを1本ずつ接続する。

なお、これは本物の銀行としては少し「お粗末」である。通信エラーやマシンの故障があったら、システムが停止するだけでなく、お金が無くなる可能性がある。分散システム上で完璧なトランザクション処理を実現するのは大変である。

6.3.5 諸注意

まず設計をする。関数を単位として、引数と戻り値の決定をする。また、外部変数を中心としてデータ設計をし、そのデータを参照する関数と修正する関数を決定する。通信アプリについては、ターゲットとホスト間にどのようなデータの通信が発生するかもまとめる。

実装において、よく分からない関数があったら、それを呼び出す簡単なプログラムを作るなどして実験して理解する。特にカーネル関連の関数は、適当に呼んで動くというものではない。一つ一つきちんと理解して進める。

これは必須ではないが、設計方針が決まったら、教員または TA にそれを説明してみるとよい。自分のシステ

ムを大雑把に人に分かるように説明できることは、とても重要である。また、根本的な間違いがあることもあるので、それを早期に知るためにもお勧めする。

7 第7回

今回の内容:

- インタビュー
- レポートの書き方
- 最終課題をまとめる

7.1 インタビュー

授業内容をどの程度理解しているかを直接質問して調べる。山崎が質問するので、暗記していればそれを答えればよい。暗記していないならば、ノート（や資料への書き込み）を見せればよい。山崎の前で実演してもよいが、試行錯誤しているようでは駄目。サッと実演できるならば、それでも良い。

7.2 デバッグ・テスト・性能測定

前回に引き続き、コーディングならびにデバッグをする。ある機能の実装が順調にできたら、別の機能にも挑戦してみる。ないしは、データ表現を工夫して高速化したり、プログラムを簡潔にしたりできないか考えてみる。

テストもすること。基本的な動作を確認したら、さまざまなケースについて動作確認する。システムコールやライブラリコールのエラーを判定していないコードは論外であり、それ以外にもできるだけエラーをチェックすべきである。テスト手順をきちんとメモしておき、最終報告書にて報告すること。

また、時間があったら性能測定もしてみると良い。特に並行プログラムはアルゴリズムを工夫すると速度が上がるはずである（測定値に反映されるほど上がらないこともある）。時間の測定は、Cの中で自分で時間測定関数（times, clock）を呼ぶ方法と、shellレベルでtimeコマンドを使う方法がある。

開発の過程を便利にする工夫ができないかも自分で考える。とにかく楽をするように考える。例えば、scpを含む一連の手順を実行する一つのコマンドを作ってみるなどである。自分が考えて行った工夫もレポートに書いて良い。

デバッグ:

GUIの整っている環境では、IDEと呼ばれる開発環境の中でデバッグができる。クロス開発のようなCUIの環境、OSやドライバの開発などではgdbと呼ばれるデバッグを使うことが多い。Raspberry Piにもgdbがインストールされているので試してみるとよい。

7.3 最終報告書

最終報告書を書く。最終報告書はtxt, pdf, docのいずれで提出すること。（txtをWindowsにコピーする場合は、geditでshift JIS, Windows改行の設定をする必要がある。授業中に指示する。）

DebianにはLibreOfficeというソフトが入っているので、これを使っても良い。docxで保存ができる。また「PDFとしてエクスポート」を使えばpdfに変換できる。

ワードを使いたい場合は、PC実習室のPCを使う。（ただし、勝手にPC演習室に行かないで、まず山崎に許可を得ること。）

整形文書の作成:

Linux において、きちんと整形された文書を書くには、TeX というソフトを使うと良い。この文書も TeX を使って書かれている。研究論文も TeX のことが多いので、大学院に進む予定の人は、知っておいて損はない。この機会に自分で TeX を勉強するのも良いだろう。(ただし、現在の設定では TeX はインストールされていない (platex を実行すると確認できる)。興味のある人は勝手にインストールしてもよい。)

最終報告書は例えば次のような章構成とする。A4 で 5 ページ前後とする。(レポートの基本的な書き方は、総合導入演習で勉強したはず。特に、「私」が文章に出てくることはありえない。また、「ですます」でなく「である」調で書くこと。)

表紙

【学籍番号，氏名，提出年月日】

1. はじめに

【このレポートの目的を書く】

2. 組み込み開発環境の概要

【今回の組み込みシステム開発のための機器，ソフト，役割，手順などについて書く】

3. 動作の概要

【どのコースについても複数のプログラム（コンポーネント）から構成されている】

【これらがどのように連携しているかの概要を書く】

【通信アプリの場合，サーバ側とクライアント側，およびそれらの中にもコンポーネントがある】

【組み込みアプリの場合，アプリ内にも入力系と出力系があり，ユーザとのインタラクションもある】

【並行プログラムの場合，2 つの銀行間および銀行内にはスレッドがある】

4. システムの詳細

【自分の作ったプログラムを掲載し，その説明を書く】

【テストについても書くこと】

【プログラム以外にもツールの使い方の工夫など，自分で考えたことを何でも書く】

【プログラムは全部載せる必要はないが，自分が手を入れた部分は載せること】

5. おわりに

【この演習で習得した知識や技術などを簡単に書く】

【この演習の感想も書く（普通の技術レポートでは感想は書かないが，これは大学の授業なので特別）】

すべてが完了したら，自分のファイルをすべて消去するため，以下を行う。まず，必要なファイルを USB メモリなどに保存する。ログアウトして，enshu でログインする。(自分がログインしたままだと自分は消去できない。)

```
# deluser --remove-home 自分のアカウント名
```

付録 A MyVolume への保存

MyVolume のアクセスを準備する

まず、AMI (<https://ami.sic.shibaura-it.ac.jp/>) にアクセスして、自分の MyVolume のサーバ名を確認しておく。どの部分がサーバ名かよく見ること (yfx3.sic.shibaura-it.ac.jp などの名前)。

以下の手順を行う。

アクティビティ→ファイル→他の場所→サーバーへ接続

空欄の中に「smb://AMI で確認したサーバ名)」を書いて「接続」をクリック

ユーザ名: 学情のアカウント

ドメイン: sic

パスワード: 学情のパスワード

左側のホスト名のところで右クリックして「ブックマークを追加」

2 回目以降は、ブックマークからすぐに開くことができる。

なお、授業資料のある ShareFolders にアクセスするには、上のサーバ名のところを「yshare.sic.shibaura-it.ac.jp」とし、フォルダを、「ShareFolders」とすれば良い。

付録 B エディタ: vi と emacs

B.1 vi

vi はファイルを修正するときに起動し、修正が終わったら vi を終了するという使い方をする。

\$ vi ファイル名

vi には、入力モードとコマンドモードという 2 つのモードがあり、今どのモードなのかを意識しなければならない。vi を立ち上げたときにはコマンドモードである。コマンドモードにおける主なキーは以下の通り。

h j k l (または←↓→↑):それぞれ左下上右の移動

x (または DELETE):一文字削除 (改行は削除できない)

dd:一行削除

a:入力モードにする (入力モードからコマンドモードにするには ESC)

i:入力モードにする (入力モードからコマンドモードにするには ESC)

ZZ:保存して終了

a と i の違いは、新たな文字を、今いる文字の右に入れるか左に入れるかである。

B.2 emacs

emacs は、vi と異なり立ち上げばなしにしておくのが普通である。

\$ emacs &

とすると、emacs 用のウィンドウが新たに作られる (&の意味は F.5 を参照)。そちらのウィンドウに移り、**^X^F** でファイルをオープンし、**^X^S** でセーブするという形でファイルを修正する。

emacs は vi と異なりモードはない。キー入力すると、いつでもそれに対応する動作をする。以下で、**^**はコントロールキーの同時押しを意味する。

^X^F:エディットしたいファイル名を指定する

^B ^P ^N ^F (または←↓→↑):それぞれ左下上右の移動

^D (または DELETE): 一文字削除 (改行も削除できる)

^X^S: 保存

^X^C: 終了 (終了する必要はない)

付録 C git

Git に関しては本が沢山出版されている。Web ページも沢山あるが、例えば <https://git-scm.com/book/ja/v2> こちらは本がそのまま Web になっており情報量が多い (多過ぎるかも)。

git reset

`git reset` オプション 1 オプション 2

と書くが、オプション 1 には次の 3 つがある

- オプションなし: commit と add を戻す (ファイルは現状のまま)
- `--soft`: commit だけを戻す (インデックスとファイルは現状のまま)
- `--hard`: commit と add とファイル修正を戻す

オプション 2 には、どのコミットまで戻るかを指定する。HEAD や HEAD^ などである。

コミットではなく、次のようにファイル名も指定できる。

```
$ git reset test.c
```

これはインデックスへの登録を取り消す。つまり `git add test.c` だけを取り消す。

git checkout

`git checkout` **ブランチ名**

と書くのが普通であるが、ブランチ名以外にコミット名も指定できる。そのコミットに戻るので、昔のファイルを見たりするのに使える。

`git checkout` **コミット名 ファイル名**

と書くと、そのコミットのファイルの状態に戻せる。これはファイルを間違っ削除した時にとても便利である。

`git reset --hard HEAD` とやると、全部のファイルが戻ってしまうが、`git checkout HEAD` **ファイル名** とすれば、そのファイルだけに戻せる。

git diff

昔のバージョンと今のバージョンを比較したいことがある。一つの方法は上で書いたように、checkout コマンドでそのバージョンに行って中身を見えることである。もう一つの方法は、

```
$ git diff HEAD^ HEAD
```

とする。これによって、現在のコミットと一つ前のコミットの差分を見ることができる。もっと昔のコミットと比較するには、`git log` でコミット一覧を見る。コミットのハッシュ値が表示されるので、その先頭の数文字を入れればコミットを直接指定できる。(この時、便利になるのはコミットメッセージだけなので、適切なメッセージを付けることが重要。)

その他

ファイルを削除したことを登録するときは、`git add`ではなく、`git rm` ファイル名 を実行する。

`git add` ファイル名の代りに、`git add -A` とすれば、管理下のすべてのファイルへの変更を一気にインデックスに登録できる。(ファイルの新規追加, 削除, 修正のすべてが対象である.)

付録 D GitHub 補足

ここでは GitHub のアカウント作成, 公開鍵の登録, GitHub 上でのリポジトリの作成について簡単に説明する。各操作の詳細な情報については GitHub の公式ドキュメント (<https://docs.github.com/ja>) を参照すること。

D.1 アカウント作成

既に GitHub のアカウントを持っている人はこの演習のために新しく GitHub アカウントを作る必要はない(作ってもよい)。

1. GitHub の Web サイト (<https://github.co.jp/>) にアクセス
2. Web ページ右上にある「サインアップ」をクリック
3. 各種情報を入力して「Create account」をクリック
 - Username: 任意の名前 (好きな名前をつける)
 - Email address: 芝浦工大のメールアドレス (推奨。別のアドレスでも構わない)
 - Password: 15 文字以上, もしくは 8 文字以上で数字と英小文字を両方含む

D.2 SSH 用公開鍵の登録

内容は配布資料「3.5 scp を便利にする」とほぼ同じ。

1. 演習用 PC でターミナルを開く
2. `ssh-keygen` で鍵ペア (公開鍵, 秘密鍵のペア) を作る

```
$ ssh-keygen -t ed25519
```

 ← ed25519 は楕円曲線暗号方式で鍵ペアを生成する
パスフレーズ (passphrase) は何も入力せず Enter キーだけを押す。
3. `~/.ssh` の下に以下のファイルが生成されているのを確認する
 - `id_ed25519`: 秘密鍵
 - `id_ed25519.pub`: 公開鍵
4. 公開鍵を GitHub に登録する
 - (a) GitHub にサインインする
 - (b) 右上のアイコンをクリックし, メニューから「Settings」を選択
 - (c) Settings 画面の左メニューから「SSH and GPG keys」を選択
 - (d) 右上の「New SSH key」ボタンをクリック
 - (e) 任意のタイトルをつける (何に使う鍵か分かりやすくしておくともよい)
 - (f) 演習用 PC のターミナルで手順 3 で作成した公開鍵を `cat` コマンド等で表示し, 内容をコピーして「Key」のテキストエリアに貼り付ける
 - (g) 「Add SSH key」ボタンを押すこれで毎回ユーザー名やパスワードを入力することなく GitHub へ push や pull ができるようになる。

D.3 リポジトリの作成

ローカルに作成した git リポジトリを GitHub のリモートリポジトリにアップロードするには予め GitHub 上にリポジトリを作成しておく必要がある。ここでは GitHub 上にリポジトリを作成する手順を紹介する。

1. GitHub にサインイン
2. 画面右上の「+」をクリックし、「New Repository」を選択
3. 以下の情報を入力して「Create repository」をクリック
 - Repository name: 任意の名前 (例: proj1)
 - Description: リポジトリの説明を書く。 (例: プロジェクト演習 14 用リポジトリ)
 - Public/Private: 「Public (デフォルト)」を選択
 - Initialize this repository with: 何もチェックしない

付録 E gnuplot

gnuplot には沢山のコマンドがあるので自分で調べる (マニュアルは 200 ページ以上あり、とても説明できない)。gnuplot に入ってからのようにすれば、簡単なヘルプ情報が出てくる。

```
gnuplot> help
```

とすればマニュアルを参照できる。help plot などとすれば、plot のマニュアルが表示される。

plot の基本的な書き方は、

```
plot {範囲指定} {関数指定 | データ指定} スタイル, {関数指定 | データ指定} スタイル, ...
```

である。範囲指定は、例えば [-3:3] [0:5] と書くと x 軸の範囲が -3~3, y 軸の範囲が 0~5 となる。

データ指定とは、データが入ったファイル名の指定であり、ダブルクォート (") で囲む。ファイルの内容は次のような形式である。

```
値1 値2 値3 値4 ...
```

一行はタブ (またはスペース) で区切った数値であり、これが一つのデータである。このような行をデータの数だけ繰返す。一つのデータのうちのどのカラムを X 軸、どれを Y 軸にするかは、ファイル名に続けて using で指定する。例えば、"a.txt" using 1:3 というデータ指定を書けば、ファイル a.txt の各行の最初の値を X 軸に、3 番目の値を Y 軸にせよという意味になる。

ファイル中のデータの順番はソートされていなくても構わないが、折れ線グラフの場合は、このデータの出現順に線を引く。

スタイルは with 種類と書く。主な種類は次の通り。

- dots: 点だけ
- points: 記号 (× など)
- lines: 折れ線グラフ
- boxes: 棒グラフ (このままだとあまり見易くない)

3 次元の曲面をファイルからデータを読み込んで書く場合は、一連のデータ群は改行だけの行で分けられるという点が重要である。

```
1900 1 降水量
...
1900 12 降水量
```

のように、1900 年のデータと 1901 年のデータの区切りを示すため、改行を挟む。

付録 F shell プログラミング

本章では、shell でのプログラミングに関して、基本的な考え方を説明する。詳細にはついては自分で調べる。

F.1 ファイル名の指定

コマンドの引数（正式にはオプション）などでファイル名を指定することがある。ファイル名の指定方法の基本は、次のような書き方である。

```
/home/yamazaki/read.c
```

これは、ルート (/) の下の home の下の yamazaki の下の read.c というファイルである。

```
D2/F
```

のように"/"で始まらないときは、今いるディレクトリが開始点となる。つまり、今のディレクトリにある D というディレクトリの下に F である。

その他、以下のような記法がある。

- ・ 今いるディレクトリ (ピリオド)
- .. 一つ上のディレクトリ (ピリオド 2 つ)
- ~ ホームのディレクトリ

複数のファイルを指定したいときには、?と*を使う。?は任意の一文字、*は任意の文字列という意味である。幾つか例を挙げる。

- * そこにあるファイルすべて
- *z z という文字で終わるファイル
- a* a という文字で始まるファイル
- *m* 中に m という文字を含むファイル
- ? 任意の 1 文字のファイル
- ??c 3 文字のファイルで最後は c で終わるファイル

F.2 コマンドの実行

コマンドを入力すると、shell はどのようにしてそれを実行するのだろうか。

```
$ echo $PATH
```

と実行すると、コロン (:) で区切られたディレクトリ名が表示される。shell は、これらのディレクトリを順番に訪問し、入力されたコマンドと同じ名前のファイルがあるかを探す。もし、あればそれを実行する。このコマンドを調べていくディレクトリ群を実行パスと呼ぶ。Linux のコマンドがどれだけあるのか知りたかったら、実行パスのディレクトリを一つ一つ ls してみる。(演習用の debian では 1899 個ある。)

なお、

```
$ /home/enshu/a.out
$ ./a.out
```

などのようにディレクトリ名も付けて入力した場合は、実行パスとは関係なく、そのファイルをそのまま実行する。

F.3 リダイレクションとパイプ

コマンドの入力と出力をキーボードや画面でなく、ファイルに対して行うことを、リダイレクションと言う。リダイレクションの一般形は、

```
$ コマンド <入力ファイル >出力ファイル
```

である。

|を使うと、コマンド₁の出力をコマンド₂の入力へリダイレクションできる。このようなリダイレクションのことをパイプと言う。リダイレクションの組合せとパイプは基本的には同じ動作だが、大きな相違点がある。それは、パイプの左側が終わらないコマンドの場合である。コマンドの中にはずっと出力をし続けるものがあり、実行が終了しないのでリダイレクションも完了しない。パイプは、データが出力される端からパイプの右側のコマンドに渡されるので、終了しないコマンドでも動作する。

```
$ コマンド1 | コマンド2
```

F.4 コマンドの便利な入力法

↑を入力すると、昔のコマンドが表示され、それを実行できることは本文で説明した。

キー入力の途中で TAB を押すと候補となるファイルを教えてくれる。候補が一つしかないときは、TAB を押すとそのままそのファイルが表示される。候補が複数あるときは、TAB を 2 回押すと一覧が表示される。(このように、途中まで入力すると後を補ってくれる機能を自動補間 (コンプリーション) と呼ぶ。)

F.5 プロセス制御

```
$ ./aaa
```

と実行すると、bash は aaa を自分の中から関数のように呼び出して、それが終了するまで待っている。もし、aaa が無限ループになってしまったりして止めたくなったときには、[^]C を押す (これは Control キーと C キーを同時に押すという意味)。

aaa を実行したまま、bash が待たないようにすることもできる。

```
$ ./aaa &
```

と実行すると、aaa は bash と並行して動作する別プロセスとして実行される。これを裏 (background) での実行という。bash は bash で動作しているので、別のコマンドを実行したりできる。裏で実行されるプロセスは、キーボードとは切り離されているので、何も入力できないし、[^]C もできない。

裏のプロセスを前面にもってくるには、fg というコマンドを使う (fg は foreground の略)。逆に bg というコマンドもある。

プロセスにシグナルを送るには、プロセス番号に対して、

```
$ kill -シグナル番号 プロセス番号
```

を実行する。プロセス番号は、ps コマンドで見ることができる。

練習: ps コマンドで bash のプロセスを見つけて、それに対して、9 番シグナルを送ってみよう。

F.6 shell スクリプト

本文で述べたように、端末アプリで手で入力する内容をそのままファイルに書けば、shell スクリプトとして実行できる。ただし、以下のことが必要である。

- スクリプトの 1 行目に `#!/bin/bash` と書く。これにより、沢山種類がある shell のどれを使うかが指定される。2 行目移行に実行したいコマンドを書く。
- `chmod a+x` などでファイルを実行可能にする。
- 一般コマンドのように使いたい場合は、`~/bin` に置く。

F.7 変数と制御

普通のプログラミング言語と同様に変数があり、繰り返し実行や条件分岐もできる。これは特に shell スクリプトを書くときに重要である。

変数への代入は `=` を使う。また、変数の値を取り出すには `$` を付ける。

```
x=3
echo $x
```

コマンドの引数に書いた値は、第 1 引数は `$1` という特殊変数で参照できる (以下同様に `$2`, `$3` である)。

```
echo $1 $2 $3
```

のようなスクリプトを作って実験してみる。

繰り返しは `for` を使う。例えば、全ファイルを表示するには、次のように入力する。ここで、`*` は正規表現で、すべてのファイルを指定している (正規表現は「データ構造とアルゴリズム」の授業で説明した)。

```
for f in *; do
    echo $f
done
```

条件分岐は `if` を使うが、特殊な書き方なので、なかなか使いにくい。ここでは例を挙げるので、後は自分で調べる。

```
if [ $x = $y ]; then
    echo same1
elif [ $x -eq $z ]; then
    echo same2
else
    echo diff
fi
```

最初の `if` は文字列としての比較、2 番目の `elif` は数値としての比較である。スペースも正確に上のように書かなければならない。

F.8 shell スクリプトの例題

shell スクリプトの勉強には、例題を読んでもるのが一番てっとり早い。Linux のコマンドの中には、shell で書かれたものが沢山ある。

```
$ cd /bin
$ file * | grep shell
```

こうすると、shell で書かれたコマンドが見つかる。`/usr/bin` や `/etc` の下などにも沢山ある。

F.9 .bashrc と alias

bash は立ち上がるときに、`~/.bashrc` というファイルを読み込む。自分独自の設定はここに書いておく。

特に重要なのは `alias` である。`alias` というコマンドを使うと、自分の好きなようにコマンドをカスタマイズできる。例えば、`.bashrc` に

```
alias ls='ls -F'
```

と書いておくと、`ls` というコマンドを入力すると、自動的に `-F` オプションが付いて実行される。

F.10 (この授業で) よく使う Linux のコマンド一覧

(*) がついているのは超重要コマンド。

- `adduser`: 新たなユーザアカウントの追加
- `alias`: 別名コマンドの設定
- `cat`: ファイルの中身を見る (*)
- `cd`: ディレクトリを移動する (*)
- `chmod`: ファイルの状態を変更する (*)
- `cp`: ファイルをコピーする (*)
- `echo`: 画面に文字を表示する
- `exit`: shell を終了する
- `file`: ファイルの種類を表示する
- `find`: ファイルを探す
- `ftp`: ファイル転送
- `gpasswd`: グループの管理をする
- `grep`: ファイルの中から文字列を探す
- `ls`: ファイルの一覧を表示する (*)
- `mkdir`: ディレクトリの作成 (*)
- `mv`: ファイルを移動する (*)
- `patch`: パッチを当てる
- `ps`: プロセスの一覧を見る (*)
- `pwd`: 今いるディレクトリの名前を表示する (*)
- `rm`: ファイルを削除する (*)
- `rmdir`: ディレクトリを削除する (*)
- `scp`: マシン間でのファイル転送
- `strace`: コマンド実行中のシステムコールを表示する
- `tar`: ファイル群を一つのファイルにする/元に戻す
- `which`: コマンドがどこにあるか教えてくれる

付録 G C プログラミング

本章では、アプリケーションを C でプログラミングする際に知っておきたい幾つかの技術を説明する。

G.1 C の超基本

C の構造体とポインタを忘れてしまった人へ。

構造体のメンバへのアクセスは、「構造体変数.メンバ名」と書く。例えば、

```
x.member
```

ポインタの宣言、ポインタの取得、ポインタの指す先への代入

```
int i, *p;
p = &i;          // p は i を指すポインタ
*p = 1;          // p の指す先に 1 を代入
*p = *p + 1;     // p の指す先の値に 1 を足して代入
```

ポインタが構造体を指している場合のメンバのアクセス

```
struct my_struct{
    int x;
};

struct my_struct *p;
p->x = 1;          // (*p).x = 1; と同じ
p->x = p->x + 1;
```

これだけでは思い出せない人は、かなりヤバイです。自習して下さい。

G.2 コマンドへの引数

```
$ ./a.out 100 200 300
```

のように、コマンドに引数を渡すことができる。これには `main` を次のように書く。

```
int main(int argc, char *argv[])
{
    printf("argc=%d,", argc);
    printf("argv[0]=%s,", argv[0]);
    printf("argv[1]=%s\n", argv[1]);
}
```

`argc` は、引数の個数+1が入っている。上の例なら4である。「`argv` は配列で、その要素はポインタであり、`char` を指す」のだった（「プログラミング」でさんざんやったはず）。つまり、`argv[0]` や `argv[1]` は文字列である。上の実行結果は、`argc=4,argv[0]=./a.out,argv[1]=100`

1 個目の引数は `argv[1]` に入っているの、これを `atoi()` に与えれば `int` になる。

G.3 システムコール/ライブラリコールのエラー

システムコール/ライブラリコールを `man` で見ていると、「エラーの理由は、`errno` に設定される」というような記述がある。システムコール/ライブラリコールでエラーが起きると、その詳細な理由が `errno` という外部変数に代入される。値の意味は、`/usr/include/asm-generic/errno-base.h` 等を書いてあるが、これをいちいち調べるのは大変である。

`perror` を使うと、見易い形でエラー理由を表示してくれる。

```
val = システムコール名(～);
if(val < 0){
    perror("コメント:");
    exit(1);
}
```

```
}
```

などとやると、とても簡単にエラーの理由が表示できる。

G.4 プロセスの作り方

プロセスの作り方は、第 1 回目に説明した通りであるが、`fork` をすると元のプロセスがコピーされて動くだけであり、まったく関係ないプログラムは実行できない。このためには、`execve` (`execl`, `execv` など) を使う。

`fork` した後で、子プロセスの中で `execve` の第 1 引数にファイル名を指定して呼び出す。すると、今のプロセスがそのファイルのプログラムで置き換わり、実行が始まる。(`execve` を実行すると、プログラムが置き換わるので、`execve` はエラーのとき以外はリターンしない。)

G.5 mmap

`mmap` は、引数も機能も多くて少しややこしいので説明する。

`mmap(addr, length, prot, flags, fd, offset)`

である。引数のデータ型は `man` で調べる。引数の意味は

`addr`: ファイルを何番地に `mmap` するか指定。 `NULL` の時は OS が適当に決める。自分で指定するときは 4096 の倍数でなければならない。

`length`: `mmap` したいメモリのサイズ (長さ)

`prot`: `protection` の略で主に使うのは次のとおり。

- `PROT_READ`: 読む
- `PROT_WRITE`: 書く
- `PROT_READ` | `PROT_WRITE`: 読み書き

なお、この指定と元のファイルを `open` した時の属性 (`O_RDONLY`, `O_WRONLY`, `O_RDWR`) が一致していないと `mmap` がエラーとなる。

`flags`: 他のプロセスと共有するか指定 (普通は `MAP_SHARED`)

- `MAP_SHARED`: 他のプロセスと共有されるし、元のファイルとも共有される (つまり書き込める)
- `MAP_PRIVATE`: 自分のプロセスだけで `mmap` する (共有メモリに書き込んでもファイルに反映されない)

`fd`: 対象となるファイル記述子。

`offset`: ファイルの先頭から `offset` バイト目から (`length` 分) を `mmap` する。 `offset` は 4096 の倍数でなければならない。

戻り値は (`void` への) ポインタで、`mmap` されたメモリ領域の先頭のアドレスである。

G.6 ソケットプログラミング

基本的な説明は本文で行った。ここでは、それ以外の点を幾つか説明する。

G.6.1 ソケットの close

ソケットはどちらから閉じてもいいが、クライアントが `close` をしてから、サーバが `close` をする方が好ましい。クライアントが `close` したことをサーバが知るには、`read` の戻り値が 0 かどうかを見ればよい。それを判定してからサーバが `close` をする。(`read` の返す値は、自分で `man` で確認してみる。)

しかし、デバッグしている最中などは、プログラムがその手順で動作するとは限らない。すると、次の `bind` がエラーとなってしまう。これは、まだそのソケットが使用中と OS が判断してしまうからである。実は、そのまま数分待つと自動的にソケットが終了するので、また使えるようになる。しかし、それではデバッグに時間がかかるので、次のようにすると良い。

```
int flag=1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &flag, sizeof(int));
bind(~);
```

setsockopt は、ソケットにオプション設定をする。上のオプション SO_REUSEADDR は、そのアドレスが使用中であっても、構わず bind せよという指示である。この後 bind を実行すれば、bind はエラーにならない。ただし、このオプションはデバッグが終わったら外した方がよい。

fork すると、ソケットもコピーされ、どのプロセスもそのソケットを読み書きできるようになる。そのようなソケットを close する時は注意が必要である。ソケットが fork によりコピーされた場合、すべてのプロセスでそのソケットを close する必要がある。一つでも残っていると、ソケットは開いたままになる。つまり、一番最後の close を実行した時に、本当の close 処理が実行される。

close は両方で実行しなければならないが、何らかのバグにより、一方だけが close した状態で、反対側から read や write をすると、次のような挙動となる。read した場合はエラーとはならず、EOF とみなされ 0 がリターンされる。write した場合は、SIGPIPE というシグナルが上がる。SIGPIPE のデフォルトの動作は単なる終了なので、何が起きたのかわからないことがよくある。この場合は、signal システムコールを使って SIGPIPE にハンドラを割り当てておくといよい。

なお close に関連して、きちんとエラー無く通信を終わる方法の一つに shutdown がある。shutdown を使うと、もう送信データがないということを明示的に相手に示すことができる。詳細は自分で調べること。

G.6.2 複数要求の受け付け

演習の例では、accept した後にすぐ sockfd を close していた。listen の引数に渡したソケットは、クライアントからの要求を受け付けるためだけのソケットである。accept は、要求があると新しくソケットを生成しそれ (例題における new_sockfd) を戻す。実際の通信は new_sockfd に対して行う。

元の sockfd はどうなったのか。実は、コネクション要求は、一つだけでなく複数受け付けることができる。先の例のように、close(sockfd); とせず、もう一度 accept を呼べば、次の要求を受け付けることができる。

G.6.3 複数のファイル記述子の待ち

accept を複数回呼び出して、複数の要求を受け付け、複数のソケットが張られたとしても、複数のマシンと通信できるわけではない。

```
while(1){
    read(new_sockfd1, buff1, 100);
    read(new_sockfd2, buff2, 100);
}
```

などとしたとすると、もし new_sockfd1 にデータがなければ、read はそこで待ちに入る。従って new_sockfd2 に来ているデータはずっと読み込まれないことになる。

これを回避する方法の 1 つは、一つのソケットに対して一つプロセス/スレッドを作り、それぞれが個別のソケットを read すれば良い。もう一つの方法は IO 多重化 poll (または select) を使う方法である。また、非同期 IO である aio_read, aio_write を使う手法もある。

G.7 get_filename

それほど難しい処理ではないが、本質とは関係ないことなので、参考として提示する。

```
// buff[] should be like "/index.html HTTP/1.1\r\n"
// get_filename extracts the file name "index.html", and
// create a new string using malloc.
```

```
// if buff=="/", return "index.html"
char *get_filename(char *buff)
{
    if (*buff == '/') {
        char *p, *fname;
        buff++;
        p = strchr(buff, ' ');
        if (p==0) { printf("Illegal format\n"); exit(1); }
        *p = '\0';
        if (p==buff) return "index.html";
        fname = malloc(strlen(buff)+1);
        strcpy(fname, buff);
        return fname;
    } else {
        printf("Unknown header %s\n", buff);
        exit(1);
    }
}
```

付録 H Raspberry Pi (Apple Pi) のプログラミング

下の基板が Raspberry Pi の本体で、上に乗っている基板が Apple Pi である。

H.1 LCD の制御

lcd.c の関数の説明

- `lcd_cmdwrite()`: I2C にコマンドを送る関数。どのようなコマンドがあるかは LCD のマニュアルに記述がある。
- `lcd_datawrite()`: I2C に表示するデータを送る関数。データは最初のバイトの RS ビット (bit6) を 1 にする必要がある。16 進で表現すると 0x40 となる。連続データ送信機能 (Co=0) を使って送っている。
- `initLCD()`: LCD の初期化。どのようなコマンドを送るかはマニュアルに書いてあるので、その通りに実行しているだけである。

LCD は横 40 ドット縦 16 ドットのディスプレイであるが、ST7032i は文字を単位として表示することしかできないため、横 8 文字縦 2 文字という粗い単位でしか表示できない。ビットマップディスプレイのように 1 ドット単位での ON/OFF はできない。

しかし、CGRAM (Character Generator RAM) という機能を使うと、オリジナルの文字を 6 文字まで定義できる。これを使うと少しだけ自由度が上がる。~/pj/devices/cgram.c は、この CGRAM を使ったプログラム例である。まずは動かしてみて欲しい。横 5 ドット縦 1 ドットの線がさまざまな位置に表示されている。

一番下の横線は'_'、真ん中の横線は'-'である。一番上の横線は文字コード 0xFF である (AQM0802.pdf を参照)。

それ以外の横線はオリジナルの文字で次のようになっている。

文字コード 1: 上, 中

文字コード 2: 上, 下

文字コード 3: 中, 下

文字コード 4: 上, 中, 下

文字コード 5: 空白

これは `cgram.c` の中の関数 `setCGRAM` を見ると（何となく）分かるだろう。

あとは 4.4 の課題で作成した任意の位置に文字を表示する方法と組み合わせれば、横線を横 8 箇所、縦 6 箇所の任意の位置に表示できる。ゲームの表示としてはかなり不十分だが、ごく簡単なものなら作れるだろう。

H.2 温度、湿度、気圧センサー

`bme280.c` には既に温度と気圧を測定する部分が書かれている。

```
temp = (data[0]<<12) | (data[1]<<4) | (data[2]>>4);
```

この意味を理解するには、BST-BME280.pdf の Table 18 (Page 25) と、Table 30 (Page 29) を理解する必要がある。 `temp_msb(data[0])` の 8bit, `temp_lsb(data[1])` の 8bit, `temp_xlsb(data[2])` の 4bit をくっつけて 20bit の値を作り出しているのが上のプログラムである。同様に、`pressure` は Table 29 に説明されているし、プログラムは既に書かれている。 `humidity` は Table 31 に記述があるが、これは `msb` と `lsb` しかないので温度や気圧よりは簡単である。自分で考えてみて欲しい。

H.3 GPIO と LED/タクトスイッチ

```
#define GPIO_BASEADDR 0x3F200000
#define GPIO_AREA_SIZE 4096
#define GPIO_GPFSEL0 0x0000 // for gpio 0..9
#define GPIO_GPSET0 0x001C // gpio 0..31
#define GPIO_GPSET1 0x0020 // gpio 32..53 (使わないはず)
#define GPIO_GPCLR0 0x0028 // gpio 0..31
#define GPIO_GPCLR1 0x002C // gpio 32..53 (使わないはず)
#define GPIO_GPLEV0 0x0034 // gpio 0..31
#define GPIO_GPLEV1 0x0038 // gpio 32..53 (使わないはず)
```

- GPFSEL0: その bit0-2 が GPIO0 の設定, bit3-5 が GPIO1 の設定のように 3bit ずつ 1 つの GPIO の設定となっている。3bit のパターンは今回使うのは次の 2 つだけであろう。
000: 入力
001: 出力
GPFSEL0 の後は GPFSEL1~GPFSEL5 が続くが、今回は使わないので上では定義していない。
- GPSET0: bit0-31 が GPIO の 0~31 番に対応し、1 を書き込むと GPIO 出力が 1 になる（出力設定済みの場合）
- GPCLR0: bit0-31 が GPIO の 0~31 番に対応し、1 を書き込むと GPIO 出力が 0 になる（出力設定済みの場合）
- GPLEV0: bit0-31 が GPIO の 0~31 番に対応し、GPIO の値を読み取れる（入力設定済みの場合）

GPIO 番号と Apple Pi のハードウェアの対応

- GPIO0,1: 外からは使えない
- GPIO2,3: I2C
- GPIO4 (入力): 赤外線受信
- GPIO5 (出力): LED1
- GPIO6 (出力): LED2
- GPIO7,8,9,10,11: SPI (たぶん使っていない)
- GPIO12: たぶん空き

- GPIO13（出力）：赤外線送信
- GPIO14,15: シリアル
- GPIO16,17: たぶん空き
- GPIO18,19,20, 21: I2S（Audio DA コンバータ）
- GPIO22～27（入力）：タクトスイッチ（たぶん茶色が 22，青が 27）
- GPIO28～53: 外からは使えない

H.4 SD カードの準備（教員に指示された時だけ）

SD カードに Raspberry Pi のブートイメージを書き込む手順を説明する．マイクロ SD カードを標準 SD カード変換アダプタにセットしノート PC に挿入する．（**変換アダプタの横のスライドを「LOCK」にしない!**）

```
$ cd ~/pj
$ umount /dev/mmcblk1p1          ← 下記参照
$ umount /dev/mmcblk1p2          ← 下記参照
$ sudo dd bs=4M if=sd.img of=/dev/mmcblk1 status=progress conv=fsync
```

上の umount で，”not mounted”というエラーメッセージが出ても気にしなくてよい．（ただし，スペルミスがないかはよく見ること．）dd は 2.0GB の sd.img をコピーするので，しばらく時間が掛かる．進捗（progress）が表示され，最後に進捗カウンタが進まなくなるが，あせらずに待つ．完了したら SD カードを一旦取り出す．

上で抜いた SD カードを再度ノート PC に挿入し，数秒待つ．

```
$ lsblk          ← 下記参照
$ cd /media/自分のアカウント/rootfs/etc
```

lsblk の結果の一番下の行の右の方に/media/**自分のアカウント**/rootfs とあることを確認する．あったら上の cd を実行する．なかったら，もう一回 SD カードを抜いて数秒待って挿入．それでもまだなかったら教員に相談．

上の cd によって，SD カードの中にある Raspberry Pi のディレクトリに移動したことになる．次のようにして IP アドレスを設定する．dhcpcd.conf をエディタで開く．

```
$ sudo gedit dhcpcd.conf
```

そして，

```
# Example static IP configuration:      ← いじらないこと
#interface eth0                          ← #を消去
#static ip_address=172.28.34.10/24       ← #を消去し IP アドレス修正
#static routers=172.28.34.1             ← #を消去
#static domain_name_servers=202.18.87.8 ← #を消去
```

という行を探し，Example～という行の下から 4 行分のコメント文字（#）を削除し，コメントで無い状態にする．次に，ip_address=の右辺の 10 の部分を Raspberry Pi の IP アドレス（IP アドレスは第 1 回の表を参照する）に設定する．エディタを終了し，次を実行．

```
$ cd
$ umount /dev/mmcblk1p1
$ umount /dev/mmcblk1p2
```

（cd で何もオプションを書かないとホームに戻る．cd ~と同じ．）umount で SD カードが取り外せる状態になる．何もエラーメッセージが出ていなかったら SD カードを取り出してよい．

付録I ソースコード

```
lcd.c
1 // write LCD for Apple Pi

2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <fcntl.h>
7 #include <errno.h>
8 #include <sys/ioctl.h>
9 #include <linux/i2c-dev.h>

10 int lcd_cmdwrite(int fd, unsigned char dat)
11 {
12     unsigned char buff[2];
13     buff[0] = 0;
14     buff[1] = dat;
15     return write(fd, buff, 2);
16 }

17 int lcd_datawrite(int fd, char dat[])
18 {
19     int len;
20     char buff[100];

21     len = strlen(dat); // don't count EOS (Null char)
22     if (len>99) {printf("too long string\n"); exit(1); }
23     memcpy(buff+1, dat, len); // shift 1 byte, ignore EOS
24     buff[0] = 0x40; // DATA Write command
25     return write(fd, buff, len+1);
26 }

27 void initLCD(int fd)
28 {
29     int i;
30     unsigned char init1[]={ 0x38, 0x39, 0x14, 0x70, 0x56, 0x6c };
31     unsigned char init2[]={ 0x38, 0x0c, 0x01 };

32     usleep(100000); // wait 100ms
33     for (i=0;i<sizeof(init1)/sizeof(unsigned char);i++) {
34         if(lcd_cmdwrite(fd, init1[i])!=2){
35             printf("internal error1\n");
36             exit(1);
37         }
38         usleep(50); // wait 50us
39     }

40     usleep(300000); // wait 300ms

41     for (i=0;i<sizeof(init2)/sizeof(unsigned char);i++) {
42         if(lcd_cmdwrite(fd, init2[i])!=2){
43             printf("internal error2\n");
44             exit(1);
45         }
46         usleep(50);
47     }
48     usleep(2000); // wait 2ms
49 }

50 int location(int fd, int y)
51 {
52     int x = 0;
53     int cmd=0x80 + y * 0x40 + x;
54     return lcd_cmdwrite(fd, cmd);
55 }

56 int clear(int fd)
57 {
58     int val = lcd_cmdwrite(fd, 1);
```

```

59         usleep(1000);           // wait 1ms
60         return val;
61     }

62 int main()
63 {
64 }

gpio.c
1  #include <errno.h>
2  #include <fcntl.h>
3  #include <linux/i2c-dev.h>
4  #include <linux/input.h>
5  #include <signal.h>
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <string.h>
9  #include <sys/mman.h>
10 #include <sys/types.h>
11 #include <termios.h>
12 #include <unistd.h>

13 // peripheral register physical address
14 #define GPIO_PHY_BASEADDR 0x3F200000
15 #define GPIO_AREA_SIZE    4096           // PAGE_SIZE
16 #define GPIO_GPFSEL0      0x0000       // for gpio 0..9, MSB 2bits are reserved
17 // omit GPFSEL1..GPFSEL5
18 #define GPIO_GPSET0       0x001C       // gpio 0..31
19 #define GPIO_GPSET1       0x0020       // gpio 32..53
20 #define GPIO_GPCLR0       0x0028       // gpio 0..31
21 #define GPIO_GPCLR1       0x002C       // gpio 32..53
22 #define GPIO_GPLEV0       0x0034       // gpio 0..31
23 #define GPIO_GPLEV1       0x0038       // gpio 32..53

24 void ledOnOff();
25 unsigned int memread(void *baseaddr, int offset);
26 void memwrite(void *baseaddr, int offset, unsigned int x);

27 void *gpio_baseaddr;

28 void ledOnOff()
29 {
30     unsigned int gpfsel0;
31     gpfsel0= memread(gpio_baseaddr, GPIO_GPFSEL0);           // get GPIO5 mode
32     gpfsel0 = gpfsel0 | (1<<15);           // 15=GPIO5*3, bit15 ON
33     memwrite(gpio_baseaddr, GPIO_GPFSEL0, gpfsel0);           // GPIO5 output mode
34     memwrite(gpio_baseaddr, GPIO_GPSET0, (1 << 5));           // GPIO5 high
35     sleep(1);
36     memwrite(gpio_baseaddr, GPIO_GPCLR0, (1 << 5));           // GPIO5 low
37 }

38 unsigned int memread(void *baseaddr, int offset)
39 {
40     unsigned int *p;
41     p = baseaddr+offset;
42     return *p;           // read memory-mapped register
43 }

44 void memwrite(void *baseaddr, int offset, unsigned int x)
45 {
46     unsigned int *p;
47     p = baseaddr+offset;
48     *p = x;           // write memory-mapped register
49 }

```